

IntellCar

Proyecto Intermodular CFGS Desarrollo Aplicaciones Web

- José Manuel Villanúa Pozo
- Juan Benítez Munoz

Centro Público Integrado de Formación Profesional
Alan Turing

Málaga Tech Park,
15 de Mayo de 2026

Sumario

0 - Preludio / Prelude

- Abstract & Keywords
- Glosario de términos

1 - Introducción y Objetivos

- 1.1 - Resumen del proyecto
- 1.2 - Motivación
- 1.3 - Objetivos

2 - Análisis del Sistema de Software

- 2.1 - Requisitos
 - 2.1.1 - Funcionales
 - 2.1.2 - No funcionales
- 2.2 - Resumen de Viabilidad
- 2.3 - Casos de uso)

3 - Diseño de la Solución

- 3.1 - Estructura general del sistema y Arquitectura
- 3.2 - Diseño BBDD
- 3.3 - Diseño de la API
- 3.4 - Identidad visual

4 - Desarrollo técnico (Core)

- 4.1 - Backend (Laravel)
 - 4.1.1 - Autenticación con Sanctum.
 - 4.1.2 - Integración de AWS SDK (Rekognition y Comprehend).
- 4.2 - Frontend (Turborepo)
 - 4.2.1 - Estructura del Monorepo (pnpm/ Turbo)
 - 4.2.2 - Experiencia de usuario con GSAP y Motion
- 4.3 - Infraestructura como Código con Terraform (IaC):
 - 4.3.1 - ¿Por qué Terraform?
 - 4.3.2 - VPC
 - 4.3.3 - EC2
 - 4.3.4 - RDS
 - 4.3.5 - S3
 - 4.3.6 - Elastic IP
 - 4.3.7 - Security Groups

5 - Seguridad, Calidad y Despliegue

- 5.1 - Moderación automática por IA
- 5.2 - Dockerización en entornos locales en desarrollo y en AWS en producción.
- 5.3 - Despliegue continuo (Deploy.yml con GitHub Actions)
- 5.4 - Agregación del Dominio Duck DNS
- 5.5 - Certificación SSL

6 - Conclusiones.

- 6.1 - Conclusiones Técnicas
- 6.2 - Conclusiones Personales (José)
- 6.3 - Conclusiones Personales (Juan)

7 - Futura Expansión del proyecto

- 7.1 - App en React Native
- 7.2 - Pasarela de pago (Stripe)
- 7.3 - Versión PRO

8 - Anexos

- 8.1 - Plan de Empresa Completo (IPE II).

0 — Preludio / Prelude

Abstract

IntellCar is a full-stack web platform designed for motor enthusiasts, combining four core features in a single ecosystem: a vehicle marketplace, a community events system (*Kdds*), a social network feed (*Universe*), and a personal virtual garage (*My Garage*). The backend is built with **Laravel 12** as a RESTful API authenticated via **Laravel Sanctum** (Bearer token), persisting data in a relational **MySQL** database with 21 domain tables. The frontend is a **Single Page Application (SPA)** developed in **Angular 19** using standalone components, organized as a **Turborepo monorepo** with **pnpm** workspaces. The infrastructure is containerized with **Docker/Docker Compose** and provisioned on **AWS** through **Terraform** scripts, enabling reproducible and scalable deployments. The API is fully documented with **Swagger/OpenAPI 3.0** through the L5-Swagger integration.

Keywords

Laravel 12 · Angular 19 · REST API · Laravel Sanctum · MySQL · Monorepo · Turborepo · pnpm · Docker · Terraform · GCP · Swagger / OpenAPI · Marketplace · Red social · Comunidad automovilística · SPA · TypeScript · PHP

Glosario de Términos

Término	Definición
API REST	Interfaz de comunicación entre cliente y servidor que sigue los principios arquitectónicos REST (stateless, recursos, verbos HTTP).
Bearer Token	Mecanismo de autenticación basado en un token opaco enviado en la cabecera HTTP <code>Authorization</code> .
CRUD	Acrónimo de las cuatro operaciones básicas sobre datos: Create, Read, Update, Delete.
Docker	Plataforma de contenedores que empaqueta una aplicación junto a sus dependencias en unidades aisladas y reproducibles.

Eloquent	ORM (Object-Relational Mapping) de Laravel que abstrae las consultas SQL mediante modelos PHP.
AWS	Amazon Web Services. Proveedor de servicios cloud donde se despliega IntellCar en producción.
Kdd	Abreviatura coloquial de "quedada". En IntellCar representa un evento automovilístico organizado por usuarios.
Laravel Sanctum	Paquete oficial de Laravel para la emisión y validación de tokens de API ligeros (Bearer).
Monorepo	Estrategia de organización de código en la que múltiples proyectos conviven en un único repositorio.
Paddock	Comunidad o estilo de vida del motor al que puede pertenecer un usuario (ej. clásicos, tuning, offroad...). En automovilismo, el paddock es la zona reservada de los equipos.
pnpm	Gestor de paquetes Node.js de alto rendimiento, usado aquí para gestionar el monorepo del frontend.
SPA	Single Page Application. Aplicación web que carga una sola vez y actualiza el contenido dinámicamente sin recargar la página.
Seeder	Script de Laravel que inserta datos de prueba en la base de datos para facilitar el desarrollo y los tests.
Soft Delete	Borrado lógico: el registro no se elimina físicamente, sino que se marca como eliminado mediante un campo de fecha (<code>onDeleteRequest</code>).
Swagger / OpenAPI	Estándar de descripción de APIs REST. Swagger UI genera una interfaz interactiva a partir de esa descripción.
Terraform	Herramienta de Infraestructura como Código (IaC) que permite definir y aprovisionar recursos cloud de forma declarativa.

Turborepo	Sistema de builds de alto rendimiento para monorepos JavaScript/TypeScript. Gestiona el grafo de dependencias entre paquetes.
------------------	---

1 — Introducción y Objetivos

1.1 Resumen del Proyecto

IntellCar es una plataforma web de comunidad automovilística desarrollada como **Proyecto Intermodular del ciclo formativo de Desarrollo de Aplicaciones Web (DAW)**, curso 2025/2026. El proyecto integra los conocimientos adquiridos a lo largo del ciclo en un producto full-stack funcional y desplegable.

La plataforma se articula en cuatro módulos principales o "distritos":

1. **Marketplace** — Sección de compraventa de vehículos con anuncios categorizados, filtros avanzados (marca, potencia, estilo de vida) y contacto directo con el vendedor.
2. **Eventos (Kdds)** — Sistema de quedadas y eventos ,gestión de aforo y asistencia.
3. **El Universo** — Feed de red social donde los usuarios publican artículos, fotos de modificaciones, noticias del sector y rutas. Incluye sistema de likes, comentarios y seguimiento entre usuarios.
4. **Mi Garaje** — Espacio personal donde cada usuario registra su historial de vehículos (actuales y pasados), personalizándolos con apodos, fotos e historias.

Tecnológicamente, el sistema se divide en:

- **Backend:** API REST en Laravel 12 con base de datos MySQL y autenticación Sanctum.
- **Frontend:** SPA en Angular 19, organizada en un monorepo con Turborepo y pnpm.
- **Infraestructura:** Contenedores Docker y despliegue cloud en AWS mediante Terraform.

1.2 Motivación

La afición al motor es uno de los hobbies con mayor arraigo en España, con millones de propietarios de vehículos, aficionados al tuning, coleccionistas de clásicos y participantes en eventos de velocidad. Sin embargo, el ecosistema digital disponible para esta comunidad presenta importantes carencias:

- Las plataformas de compraventa generalistas (Wallapop, Milanuncios, Coches.net) no ofrecen contexto comunitario ni segmentación por estilo de vida del motor.
- Las redes sociales genéricas (Instagram, Facebook) carecen de funcionalidades específicas para el sector: ficha técnica de vehículos, organización de quedadas, garaje virtual.

- No existe ninguna plataforma que **unifique en un único ecosistema** la compraventa, la socialización, los eventos y la gestión del historial de vehículos propios.

Adicionalmente, desde el punto de vista formativo, el proyecto representa una oportunidad ideal para aplicar en un contexto real las tecnologías impartidas en el ciclo: arquitectura cliente-servidor, diseño de bases de datos relacionales, desarrollo de APIs REST, frameworks modernos de frontend, autenticación, seguridad, despliegue en contenedores y aprovisionamiento de infraestructura AWS.

1.3 Objetivos

Objetivos Generales

- Desarrollar una plataforma web full-stack funcional orientada a la comunidad del motor, que sirva como demostración integral de las competencias adquiridas durante el ciclo DAW.
- Producir un software de calidad, documentado, testado y desplegable en entornos reales.

Objetivos Específicos

ID	Objetivo
OBJ-01	Diseñar e implementar una base de datos relacional MySQL con más de 20 tablas que modele el dominio completo de IntellCar.
OBJ-02	Desarrollar una API REST con Laravel 12 que exponga todos los recursos del sistema con separación de rutas públicas y protegidas.
OBJ-03	Implementar un sistema de autenticación seguro basado en Laravel Sanctum con tokens Bearer.
OBJ-04	Construir un SPA con Angular 19 (standalone components) que consuma la API y ofrezca una experiencia de usuario fluida.
OBJ-05	Organizar el frontend en una arquitectura monorepo (Turborepo + pnpm) que facilite la escalabilidad y reutilización de componentes.
OBJ-06	Documentar la API con Swagger/OpenAPI 3.0 mediante L5-Swagger, generando una interfaz interactiva.

OBJ-07	Implementar el módulo Marketplace con publicación, búsqueda, filtrado y gestión de anuncios de vehículos.
OBJ-08	Implementar el módulo social (El Universo) con posts, likes, comentarios y sistema de seguimiento.
OBJ-09	Implementar el módulo de eventos (Kdds) con geolocalización y gestión de asistencia.
OBJ-10	Implementar el Garaje Virtual con historial de vehículos por usuario.
OBJ-11	Contenerizar la aplicación completa con Docker y Docker Compose para garantizar entornos reproducibles.
OBJ-12	Provisionar la infraestructura de producción en AWS mediante Terraform.

2 — Análisis del Sistema de Software

2.1 Requisitos

2.1.1 Requisitos Funcionales

ID	Requisito	Módulo
RF-01	El sistema permitirá el registro de nuevos usuarios con nombre, email, teléfono, contraseña y paddock.	Autenticación
RF-02	El sistema permitirá el inicio de sesión mediante email y contraseña, devolviendo un token Bearer.	Autenticación
RF-03	El sistema permitirá el cierre de sesión invalidando el token activo.	Autenticación
RF-04	Un usuario autenticado podrá consultar y editar sus datos de perfil (nombre, teléfono, email de contacto, foto).	Perfil

RF-05	Un usuario podrá solicitar la eliminación lógica (soft delete) de su propia cuenta.	Perfil
RF-06	Un usuario autenticado podrá añadir, editar y eliminar vehículos de su garaje virtual.	Garaje
RF-07	Cualquier visitante podrá consultar el listado de anuncios del marketplace con filtros.	Marketplace
RF-08	Cualquier visitante podrá ver el detalle de un anuncio de vehículo.	Marketplace
RF-09	Un usuario autenticado podrá crear, editar y solicitar el borrado de sus propios anuncios.	Marketplace
RF-10	Un administrador podrá eliminar definitivamente cualquier anuncio.	Marketplace
RF-11	Cualquier visitante podrá consultar el feed de posts del Universo.	Social
RF-12	Un usuario autenticado podrá crear posts con texto y multimedia.	Social
RF-13	Un usuario autenticado podrá dar/quitar "like" a cualquier post.	Social
RF-14	Un usuario autenticado podrá comentar posts.	Social
RF-15	Un usuario autenticado podrá seguir y dejar de seguir a otros usuarios.	Social
RF-16	Cualquier visitante podrá consultar el listado de eventos (Kdds).	Eventos
RF-17	Un usuario autenticado podrá crear, editar y eliminar sus propios eventos.	Eventos
RF-18	Un usuario autenticado podrá inscribirse y desinscribirse en eventos.	Eventos

RF-19	El sistema clasificará a los usuarios según su rol: <code>admin</code> , <code>pro</code> (profesional), <code>indv</code> (individual) y <code>press</code> (prensa).	Usuarios
RF-20	Un administrador podrá listar todos los usuarios y eliminar cuentas de forma definitiva.	Admin
RF-21	El sistema enviará notificaciones internas a los usuarios ante interacciones relevantes.	Notificaciones
RF-22	Un usuario podrá guardar búsquedas de vehículos para consultarlas posteriormente.	Búsquedas
RF-23	Los Paddocks permitirán agrupar a usuarios, anuncios y eventos por estilo de vida.	Comunidad

2.1.2 Requisitos No Funcionales

ID	Requisito	Categoría
RNF-01	Las respuestas de la API no superarán los 300 ms en condiciones normales de carga.	Rendimiento
RNF-02	Las contraseñas se almacenarán siempre cifradas con el algoritmo bcrypt (factor de coste 12).	Seguridad
RNF-03	Todos los endpoints que modifiquen datos requerirán autenticación mediante Bearer Token (Sanctum).	Seguridad
RNF-04	Las rutas administrativas estarán protegidas adicionalmente por el middleware <code>role:admin</code> .	Seguridad
RNF-05	La aplicación deberá estar operativa en los principales navegadores modernos (Chrome, Firefox, Edge, Safari).	Compatibilidad
RNF-06	La interfaz de usuario adoptará un diseño responsive adaptado a escritorio y dispositivos móviles.	Usabilidad

RNF-07	El código del backend seguirá la arquitectura MVC y los principios SOLID .	Mantenibilidad
RNF-08	La API estará documentada con el estándar OpenAPI 3.0 y accesible vía Swagger UI.	Documentación
RNF-09	La aplicación se contenerizará con Docker para garantizar entornos reproducibles entre desarrollo y producción.	Portabilidad
RNF-10	La infraestructura de producción se definirá como código (laC) mediante Terraform sobre AWS.	Escalabilidad
RNF-11	Los listados de recursos paginados devolverán un máximo de 15 registros por página por defecto.	Rendimiento
RNF-12	Los archivos multimedia subidos por los usuarios se validarán en tipo y tamaño antes de ser almacenados.	Seguridad

2.2 Resumen de Viabilidad

IntellCar es una **startup tecnológica española del sector de la automoción** cuyo objeto social es el desarrollo y la explotación de una plataforma digital 360° para entusiastas del motor. A continuación se recoge un resumen ejecutivo del plan de empresa elaborado como parte del módulo IPE del proyecto intermodular.

Identidad y propuesta de valor

El nombre **IntellCar** —fusión de *Intelligent* y *Car*— transmite tecnología y pasión por el motor. Su eslogan es *"Todo lo que tiene motor, cobra vida."*

La propuesta de valor diferencial reside en integrar en una única plataforma cuatro funcionalidades que los competidores (Coches.net, Wallapop, motor.es) ofrecen de forma fragmentada: compraventa de vehículos, organización de eventos, red social especializada y garaje personal. IntellCar cubre el *customer journey* completo del aficionado al motor.

Segmentos de clientes

Segmento	Perfil
----------	--------

Particulares	Personas que compran o venden su vehículo de segunda mano
Entusiastas (<i>petrolheads</i>)	Aficionados que buscan comunidad, eventos y contenido de motor
Concesionarios y <i>dealers</i>	Profesionales que necesitan visibilidad digital
Creadores de contenido	Periodistas, <i>influencers</i> y canales especializados en motor

Modelo de negocio (fuentes de ingresos)

- **Freemium Marketplace:** anuncios básicos gratuitos; anuncios destacados de pago (posición *top*, badge *Pro Dealer*).
- **Suscripción Premium:** garaje ilimitado, estadísticas de anuncios y experiencia sin publicidad.
- **Publicidad segmentada:** banners de marcas automovilísticas dirigidos por Paddock (perfil de usuario).
- **Comisión por lead:** por cada contacto generado entre comprador y vendedor profesional.
- **Patrocinio de Kdds:** marcas patrocinan quedadas a cambio de visibilidad.

Análisis de mercado

El mercado objetivo principal es España, con aproximadamente **2 millones de transacciones de segunda mano al año** (DGT, 2024). No existe ningún competidor que integre las cuatro funcionalidades de IntellCar, lo que constituye su principal ventaja competitiva.

Fortalezas

- Plataforma integral (4 módulos)
- API REST escalable con Laravel
- Comunidad y fidelización
- Backend ya construido y documentado

Debilidades

- Sin tracción inicial de usuarios
- Equipo reducido (2 personas)
- Presupuesto de marketing limitado

Oportunidades

- Mercado de segunda mano en auge
- Tendencia a comunidades de nicho
- Monetización de eventos y creadores

Amenazas

- Competidores con grandes recursos
- Cambios regulatorios (RGPD)
- Coste elevado de captación de usuarios

Producto — los cuatro distritos

Distrito	Descripción	Monetización
Marketplace	Compraventa de vehículos filtrada por Paddocks (estilos de vida)	Anuncios destacados, comisión <i>lead</i>
Eventos (Kdds)	Organización y asistencia a quedadas con geolocalización	Patrocinios de marca
El Universo	Red social de contenido motor (posts, likes, follows)	Publicidad segmentada
Mi Garaje	Vitrina personal de vehículos actuales y pasados	Suscripción Premium

Estrategia de lanzamiento (*Go-To-Market*)

- **Fase Beta (meses 1-2):** invitación a 200 *early adopters* de comunidades existentes (foros y grupos de motor).
- **Fase Crecimiento (meses 3-6):** campañas en redes sociales con contenido generado por usuarios (UGC); colaboración con *influencers* del motor en Instagram, TikTok y YouTube.
- **Fase Monetización (mes 7+):** activación de suscripciones Premium y acuerdos comerciales con *dealers*.

Forma jurídica y aspectos legales

Se constituirá como **Sociedad de Responsabilidad Limitada (S.L.)**, con capital social mínimo de 3.000 € distribuido al 50 % entre los dos socios fundadores. La marca *IntellCar* se registrará en la OEPM en las clases 42 (servicios tecnológicos) y 35

(publicidad/marketplace). La plataforma cumple con el **RGPD** y la LOPDGDD: autenticación segura con Laravel Sanctum, hash de contraseñas y rutas protegidas por *middleware*.

Estimación financiera — Año 1

Concepto	Importe
Anuncios destacados (200 uds/mes × 9,99 €)	23.976 €
Suscripciones Premium (100 uds/mes × 4,99 €)	5.988 €
Publicidad (banners)	3.000 €
Total ingresos estimados	~33.000 €
Hosting / infraestructura cloud	1.800 €
Constitución S.L. + registro de marca	600 €
Marketing digital	3.000 €
Herramientas y licencias	600 €
Total gastos estimados	~6.000 €
Resultado estimado año 1	~27.000 €

Archivo Adjunto:

[Plan de Empresa - IntellCar](#)

2.3 Casos de Uso

Se identifican tres actores principales en el sistema:

- **Usuario Anónimo:** Visitante no autenticado. Puede consultar contenido público.
- **Usuario Autenticado:** Cuenta registrada y con token válido. Puede interactuar con la plataforma.
- **Administrador:** Usuario con rol `admin`. Tiene acceso a la gestión total del contenido y usuarios.

Diagrama de Casos de Uso

El diagrama UML formal de casos de uso se incluye como figura en el documento PDF adjunto. A continuación se detalla la relación actor–caso de uso.

Acciones disponibles para el Usuario Anónimo:

- UC-01 — Registrarse
- UC-02 — Iniciar sesión
- UC-03 — Ver listado y detalle de anuncios (Marketplace)
- UC-04 — Ver feed y detalle de posts (El Universo)
- UC-05 — Ver listado y detalle de eventos (Kdds)

Acciones disponibles para el Usuario Autenticado (extiende las anteriores):

- UC-06 — Gestionar perfil (ver, editar datos y foto)
- UC-07 — Gestionar garaje virtual (añadir, editar, eliminar vehículos)
- UC-08 — Publicar y gestionar sus anuncios
- UC-09 — Publicar posts con texto y multimedia
- UC-10 — Dar like y comentar posts
- UC-11 — Seguir y dejar de seguir a otros usuarios
- UC-12 — Crear eventos (Kdds) y unirse a los existentes
- UC-13 — Cerrar sesión

Acciones exclusivas del Administrador (extiende las anteriores):

- UC-14 — Listar todos los usuarios de la plataforma
- UC-15 — Eliminar definitivamente una cuenta de usuario
- UC-16 — Eliminar definitivamente cualquier anuncio
- UC-17 — Moderar y gestionar contenido (posts, eventos)

Descripción de Casos de Uso Principales

UC-01 — Registrarse

- **Actor:** Usuario Anónimo
- **Precondición:** El email y el teléfono no están registrados en el sistema.
- **Flujo principal:** El usuario completa el formulario de registro → el sistema valida los datos → crea la cuenta → devuelve el token de acceso.
- **Postcondición:** El usuario queda autenticado con un token Bearer activo.

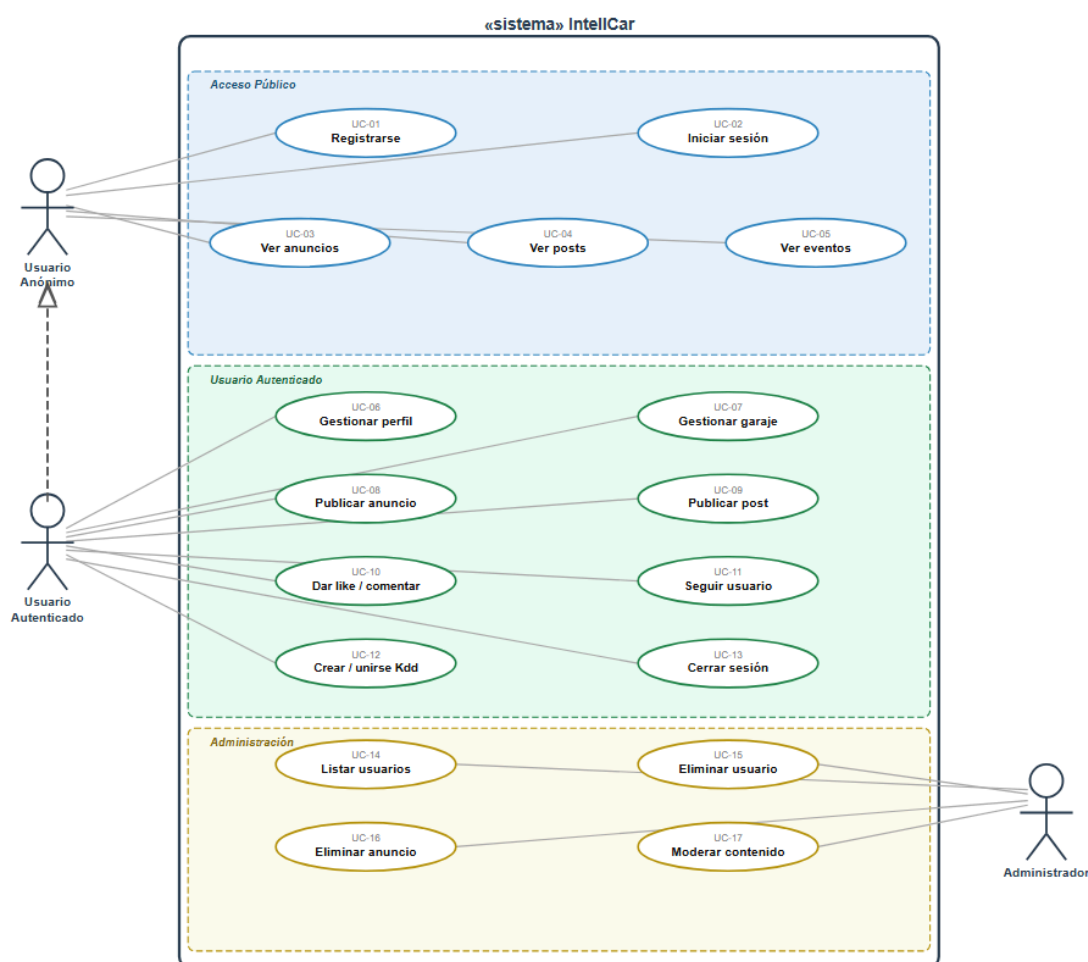
UC-08 — Publicar Anuncio

- **Actor:** Usuario Autenticado

- **Precondición:** El usuario tiene sesión activa y dispone de los datos del vehículo (marca, modelo, motor, precio...).
- **Flujo principal:** El usuario rellena el formulario del anuncio → adjunta imágenes → envía la publicación → el sistema valida y crea el anuncio (inicialmente con `visible = false` hasta revisión).
- **Postcondición:** El anuncio aparece en el marketplace.

UC-12 — Crear / Unirse a Kdd

- **Actor:** Usuario Autenticado
- **Flujo crear:** El usuario define título, descripción, fecha, ubicación y aforo → el sistema crea el evento vinculado al creador.
- **Flujo unirse:** El usuario visualiza un evento → pulsa "Asistir" → el sistema registra su asistencia si hay plazas disponibles.



3 — Diseño de la Solución

3.1 Estructura General del Sistema y Arquitectura

IntellCar sigue una arquitectura **cliente-servidor** desacoplada con tres capas diferenciadas:

Capa	Componentes
Cliente (Browser)	Angular 19 SPA — Turborepo Monorepo (pnpm) apps/browser-client · packages/ui · packages/ts-config Comunicación: HTTP/HTTPS + Bearer Token
Backend (Laravel 12)	routes/api.php → Controllers → Models (Eloquent) → MySQL Middlewares: auth:sanctum, role:admin Storage: ficheros locales / Cloud Storage Docs: Swagger UI en /api/documentation
Infraestructura	Docker / Docker Compose (desarrollo y producción) Terraform → AWS(Cloud Run · Cloud SQL · Cloud Storage)

Stack Tecnológico

Capa	Tecnología	Versión
Frontend Framework	Angular	19
Frontend Language	TypeScript	~5.x
Frontend Styling	Tailwind CSS	3.x
Monorepo Manager	Turborepo + pnpm	Latest
Backend Framework	Laravel	12
Backend Language	PHP	8.3
ORM	Eloquent	(Laravel 12)
Autenticación	Laravel Sanctum	^4.0

Base de Datos	MySQL	8.x
API Docs	L5-Swagger (OpenAPI 3.0)	^11.0
Contenedores	Docker + Docker Compose	Latest
IaC	Terraform	Latest
Cloud	Amazon Web Services	—

Estructura del Frontend (Monorepo)

```
front/
├── apps/
│   ├── browser-client/      ← SPA Angular 19 (aplicación principal)
│   │   ├── src/app/
│   │   │   ├── core/        (guards, interceptors, servicios globales)
│   │   │   ├── features/     (módulos lazy: marketplace, social, kdds...)
│   │   │   └── shared/       (componentes reutilizables)
│   │   └── tailwind.config.js
│   └── packages/
│       ├── ui/              ← Librería de componentes compartidos
│       ├── eslint-config/
│       └── typescript-config/
```

Estructura del Backend (Laravel 12)

```
src/
├── app/
│   ├── Http/
│   │   ├── Controllers/Api/  (AuthController, MarketControl, UnivControl...)
│   │   ├── Middleware/       (RoleMiddleware)
│   │   └── Requests/         (Form Requests con validación)
│   ├── Models/               (16 modelos Eloquent)
│   └── OpenApi/               (schemas Swagger)
├── routes/
│   ├── api.php                (rutas públicas y protegidas)
│   └── auth.php
└── database/
    ├── migrations/            (28 migraciones)
    ├── factories/             (7 factories)
    └── seeders/
```

3.2 Diseño de la Base de Datos

La base de datos está compuesta por **21 tablas de dominio** más las tablas de infraestructura de Laravel (`cache`, `jobs`, `personal_access_tokens`).

Diagrama Entidad-Relación

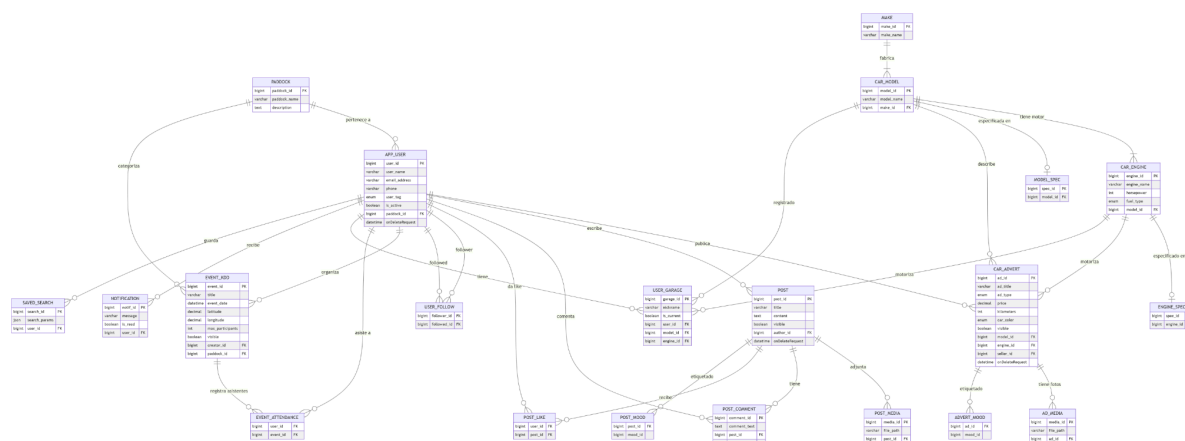
Relaciones principales:

- Un **Paddock** agrupa a muchos **usuarios** y muchos **eventos**.
- Un **app_user** puede publicar muchos **anuncios**, escribir muchos **posts**, crear muchos **eventos** y tener muchos **vehículos en garaje**.
- Una **marca (make)** tiene muchos **modelos (car_model)**; un modelo tiene muchos **motores (car_engine)**.
- Un **anuncio (car_advert)** referencia un modelo, un motor y un vendedor; puede tener muchas **fotos (ad_media)**.
- Un **post** puede tener muchos **likes**, **comentarios** y **archivos multimedia**.
- Los usuarios se siguen entre sí mediante la tabla pivote **user_follow**.
- La asistencia a eventos se gestiona mediante la tabla pivote **event_attendance**.

Descripción de Tablas

Tabla	Descripción
<code>paddock</code>	Categorías de estilo de vida (clásicos, tuning, offroad...).
<code>app_user</code>	Usuarios de la plataforma. Soporta roles (admin/pro/indv/press) y soft delete.
<code>make</code>	Marcas de vehículos (Toyota, BMW, Ferrari...).
<code>car_model</code>	Modelos de vehículo, referenciados a su marca.
<code>car_engine</code>	Motorizaciones disponibles por modelo (cilindrada, CV, combustible).
<code>car_advert</code>	Anuncios del marketplace. Referencia modelo, motor y vendedor.
<code>ad_media</code>	Imágenes asociadas a un anuncio.
<code>post</code>	Publicaciones de la red social (El Universo).
<code>post_media</code>	Archivos multimedia de un post.

<code>post_like</code>	Tabla pivote N:M entre usuarios y posts (likes).
<code>post_comment</code>	Comentarios de usuarios en posts.
<code>user_follow</code>	Tabla pivote N:M de seguimiento entre usuarios.
<code>event_kdd</code>	Eventos/quedadas con soporte de geolocalización y aforo máximo.
<code>event_attendance</code>	Tabla pivote N:M de asistencia de usuarios a eventos.
<code>user_garage</code>	Vehículos del garaje virtual de cada usuario.
<code>notification</code>	Notificaciones internas del sistema.
<code>saved_search</code>	Búsquedas guardadas por usuarios, almacenadas como JSON.
<code>model_spec</code>	Especificaciones técnicas extendidas del modelo de vehículo.
<code>engine_spec</code>	Especificaciones técnicas extendidas del motor.
<code>advert_mood</code>	Etiquetas de estado/estilo de un anuncio (pivote).
<code>post_mood</code>	Etiquetas de estado/estilo de un post (pivote).



3.3 Diseño de la API

La API REST de IntellCar está organizada en **6 distritos** que agrupan los recursos por dominio funcional. La URL base es `/api/`.

La documentación interactiva completa está disponible en:

- **Desarrollo:** <http://localhost:8000/api/documentation>
- **Producción (Docker):** <https://intellcar2.duckdns.org/api/documentation>

Distrito 0 — Autenticación

Método	Endpoint	Descripción	Protección
POST	<code>/api/auth/register</code>	Registro de nuevo usuario	Pública
POST	<code>/api/auth/login</code>	Inicio de sesión → devuelve token	Pública
GET	<code>/api/auth/me</code>	Perfil del usuario autenticado	auth:sanctum
POST	<code>/api/auth/logout</code>	Invalida el token actual	auth:sanctum

Distrito 1 — Administración de Usuarios

Método	Endpoint	Descripción	Protección
GET	<code>/api/users</code>	Lista todos los usuarios	admin
GET	<code>/api/users/{id}</code>	Perfil de un usuario	auth:sanctum
DELETE	<code>/api/users/{id}</code>	Borrado definitivo de usuario	admin
POST	<code>/api/users/{id}/follow</code>	Toggle seguir / dejar de seguir	auth:sanctum

Distrito 2 — Perfil Propio y Garaje

Método	Endpoint	Descripción	Protección
GET	<code>/api/profile</code>	Ver datos propios	auth:sanctum

PUT	/api/profile	Editar datos propios	auth:sanctum
POST	/api/profile/picture	Actualizar foto de perfil	auth:sanctum
PATCH	/api/profile/soft-delete	Solicitar borrado de cuenta	auth:sanctum
POST	/api/profile/garage	Añadir vehículo al garaje	auth:sanctum
POST	/api/profile/garage/{id}	Editar vehículo del garaje	auth:sanctum
DELETE	/api/profile/garage/{id}	Eliminar vehículo del garaje	auth:sanctum

Distrito 3 — Market (Marketplace)

Método	Endpoint	Descripción	Protección
GET	/api/market	Listado de anuncios (con filtros)	Pública
GET	/api/market/{id}	Detalle de anuncio	Pública
PUT / PATCH	/api/market/{id}	Editar anuncio (dueño o admin)	auth:sanctum
PATCH	/api/market/{id}/soft-delete	Solicitar borrado de anuncio	auth:sanctum (dueño)
DELETE	/api/market/{id}	Borrado definitivo de anuncio	admin

Distrito 4 — Social (El Universo)

Método	Endpoint	Descripción	Protección
GET	/api/social	Feed de posts públicos	Pública
GET	/api/social/{id}	Detalle de post	Pública

POST	/api/social/{id}/like	Toggle like en un post	auth:sanctum
POST	/api/social/{id}/comment	Añadir comentario	auth:sanctum

Distrito 5 — Eventos (Kdds)

Método	Endpoint	Descripción	Protección
GET	/api/kdds	Listado de eventos	Pública
GET	/api/kdds/{id}	Detalle de evento	Pública

Convenciones de la API

- **Formato de respuesta:** JSON en todos los endpoints.
- **Autenticación:** Cabecera `Authorization: Bearer <token>`.
- **Paginación:** Los listados devuelven paginación con `data`, `current_page`, `last_page` y `total`.
- **Soft Delete:** Los recursos marcados con `onDeleteRequest` o `visible = false` se excluyen automáticamente de los listados públicos.
- **Errores:** Códigos HTTP estándar (400, 401, 403, 404, 422, 500) con mensajes descriptivos en español.

3.4 Identidad Visual

IntellCar adopta una identidad visual cuidada orientada a la comunidad del motor, con una estética moderna, oscura y de alto contraste que evoca la cultura del asfalto y la velocidad.

A nivel técnico, la interfaz se implementa con **Tailwind CSS** como motor de estilos, apoyado en el sistema de componentes de la librería `packages/ui` del monorepo, lo que garantiza coherencia visual en toda la aplicación y facilita la evolución del diseño de forma centralizada.

4 — Desarrollo Técnico (Core)

4.1 Backend (Laravel)

Autenticación con Sanctum.

Para proteger los datos y asegurar que solo los usuarios registrados puedan interactuar con las funcionalidades de Intelcar, se ha implementado un sistema híbrido de autenticación. El backend de la aplicación, desarrollado en Laravel, gestiona la seguridad diferenciando de manera estricta entre el tráfico web tradicional y el consumo de recursos a través de la API REST.

1. Autenticación de la API basada en Tokens (Laravel Sanctum)

Para dar servicio a nuestra aplicación cliente (Frontend en Angular), se ha integrado Laravel Sanctum. Este paquete oficial proporciona un sistema de autenticación ligero y robusto mediante tokens de texto plano (Bearer Tokens), ideal para APIs REST desacopladas.

El flujo de control de acceso implementado en el controlador de la API (Api\AuthController) consta de los siguientes pilares:

Registro de Usuarios (register): Cuando un nuevo conductor o entusiasta se registra en la plataforma, el sistema valida rigurosamente los datos (comprobando que el correo electrónico y el teléfono sean únicos). La contraseña se cifra mediante el algoritmo hash Bcrypt utilizando la fachada Hash::make() antes de almacenarse en la base de datos RDS. Acto seguido, se emite automáticamente un token de acceso (auth-token).

Inicio de Sesión e Integridad (login): El proceso comprueba las credenciales introducidas contra los registros persistentes. Además de la verificación estándar, se ha implementado una capa de lógica de negocio adicional: el sistema comprueba el estado lógico de la cuenta (\$user->is_active). Si un usuario ha sido desactivado por la administración de Intelcar, el backend deniega el acceso devolviendo un código de estado HTTP 403 (Prohibido), previniendo accesos no autorizados.

Cierre de Sesión Seguro (logout): Siguiendo el principio de revocación inmediata, cuando el usuario decide salir de la aplicación, el servidor destruye el token de acceso activo en la base de datos (currentAccessToken()->delete()). De este modo, el token queda completamente invalidado y no puede ser reutilizado en peticiones futuras.

Sesión Persistente del Cliente (me): Se proporciona un endpoint seguro protegido bajo el middleware de Sanctum para que el cliente Angular, enviando el token en las cabeceras HTTP (Authorization: Bearer <token>), pueda recuperar en cualquier momento el perfil del usuario autenticado junto con sus relaciones lógicas, como el estado de su garaje o paddock.

2. Documentación de la API (OpenAPI / Swagger)

Con el fin de seguir un estándar profesional de desarrollo que facilite la integración y pruebas del equipo de frontend, todo el controlador de autenticación de la API ha sido documentado utilizando anotaciones de OpenAPI / Swagger.

Mediante los bloques de comentarios `@OA\Post` y `@OA\Response`, se definen explícitamente las rutas del servidor, los datos requeridos en formato JSON (como ejemplos y tipos de datos), los esquemas de seguridad basados en portadores de tokens (`securityScheme="sanctum"`) y los posibles códigos de respuesta del servidor (200 para éxitos, 401 para credenciales incorrectas, 422 para fallos de validación, etc.). Esto permite generar de forma automatizada una interfaz interactiva de pruebas para los desarrolladores.

3. Autenticación Web y Control de Abuso (Rate Limiting)

De forma paralela, el sistema hereda los controladores de sesión tradicionales (`LoginRequest` y `AuthenticatedSessionController`) para el panel de administración web o vistas del servidor. Esta capa cuenta con dos mecanismos defensivos críticos frente a ataques informáticos:

Protección de Sesiones: El método `session()->regenerate()` se dispara tras cada inicio de sesión exitoso. Esto previene ataques de fijación de sesión, asegurando que el identificador de la cookie del navegador cambie por completo una vez que el usuario se ha identificado.

Limitador de Tasa de Peticiones (Rate Limiting): Para evitar ataques de fuerza bruta que intenten adivinar contraseñas mediante scripts masivos, la clase `LoginRequest` utiliza el `RateLimiter` de Laravel. Se ha configurado un límite estricto de 5 intentos de acceso por combinación de correo electrónico e IP. Si un cliente supera este umbral, el sistema bloquea temporalmente la ruta, lanza un evento de bloqueo (`Lockout`) y calcula de forma matemática el tiempo de penalización restante, notificando al usuario mediante una excepción de validación los minutos u horas que debe esperar antes de reintentarlo.

Integración de AWS SDK (Rekognition y Comprehend).

Para garantizar la seguridad, la idoneidad de los contenidos y automatizar la categorización dentro de la comunidad de Intellcar, el backend delega el análisis avanzado de archivos y textos en dos servicios de Inteligencia Artificial basados en modelos preentrenados de aprendizaje profundo (Deep Learning): Amazon Rekognition y Amazon Comprehend.

1. Modelos Fundacionales y Entrenamiento de AWS

Antes de analizar la implementación del código, es imprescindible comprender el núcleo tecnológico que sustenta estas herramientas:

Amazon Rekognition (Visión por Computador): Está construido sobre modelos de redes neuronales convolucionales profundas (Deep CNN). Amazon ha entrenado este modelo utilizando miles de millones de imágenes y vídeos a gran escala. Gracias a este entrenamiento masivo, el modelo es capaz de extraer características complejas de los objetos (formas, texturas, geometrías) y contextualizarlas, logrando reconocer conceptos abstractos (como discernir si un coche es un "sedán" o un "SUV") con un porcentaje de precisión que supera la visión humana estándar.

Amazon Comprehend (Procesamiento del Lenguaje Natural): Utiliza arquitecturas avanzadas de aprendizaje profundo orientadas al procesamiento de texto (redes de tipo Transformer). Ha sido preentrenado con corpus lingüísticos gigantescos y multilingües (páginas web, literatura, artículos y transcripciones comerciales). El modelo no realiza una simple búsqueda de palabras en un diccionario; es capaz de entender la semántica, el contexto sintáctico, deducir la intención implícita del redactor y extraer patrones de comportamiento conversacional.

2. Implementación de Visión Inteligente (RekoControl)

El controlador RekoControl gestiona el ciclo de vida de los archivos multimedia subidos por los usuarios a través de tres flujos de ejecución asíncronos:

A. Generación de URLs Firmadas (presigned)

Para evitar que los pesados flujos de subida de imágenes pasen por el servidor EC2 y saturen su ancho de banda, se implementa el método presigned. Laravel negocia directamente con el SDK de S3 un comando seguro de tipo PutObject y le devuelve al frontend de Angular una URL pre-firmada con una validez estricta de 20 minutos. Así, el cliente Angular sube la foto directamente al almacenamiento de AWS de forma segura y eficiente.

B. Moderación de Contenido y Clasificación de Vehículos (analyze)

Cuando se procesa la imagen, se realizan de forma consecutiva tres llamadas al API de Rekognition:

Moderación de contenido (detectModerationLabels): Analiza si la imagen infringe las normas de la comunidad (por ejemplo, contenido explícito o violento) con un umbral de confianza mínimo del 50%. Si se detecta una infracción, la imagen se rechaza devolviendo un código HTTP 422.

Detección de Etiquetas (detectLabels): Se extraen hasta 50 etiquetas con un 65% de confianza mínima. El controlador filtra los resultados usando una constante restrictiva (REQUIRED_VEHICLE_LABELS). Si la IA no detecta conceptos como Car, Vehicle o Automobile, el sistema asume que el usuario está intentando subir una imagen fraudulenta o ajena a la temática de automoción de Intellcar y bloquea la publicación.

Análisis de Texto en Imagen (detectText): Extrae caracteres tipográficos incrustados en la foto (con más del 80% de confianza). Esto se utiliza estratégicamente para detectar las insignias físicas de los vehículos, lo que ayuda a cruzar los datos semánticos con los modelos de la base de datos (CarModel) para intentar predecir automáticamente la marca (Make) y el modelo exacto del coche subido.

3. Implementación de Análisis del Lenguaje (ComprehendControl)

El controlador ComprehendControl se encarga de auditar las descripciones de los anuncios de compraventa y el contenido de los posts en el foro de la comunidad mediante el método analyzeText:

Identificación del Idioma Dominante (detectDominantLanguage): Determina el lenguaje en el que escribe el usuario. Si detecta un idioma extranjero con una certeza superior al 80%, el sistema acepta el texto pero adjunta un mensaje de advertencia (Warning) sugiriendo al usuario traducirlo al español para maximizar el alcance de su publicación.

Protección de la Privacidad / PII (detectPiiEntities): Con el fin de cumplir con normativas de privacidad y evitar transacciones fraudulentas fuera de la plataforma, el sistema escanea el texto en busca de información de identificación personal (Personally Identifiable Information). Si la IA halla datos de tipo EMAIL, PHONE o ADDRESS con más de un 80% de fiabilidad, restringe el envío de forma inmediata protegiendo la seguridad de los usuarios.

Análisis de Contenido Tóxico (detectToxicContent y detectSentiment): Examina las cadenas en busca de insultos, incitación al odio o amenazas. Ante la ausencia o limitaciones geográficas de la API de toxicidad avanzada, el controlador utiliza como fallback el análisis de sentimiento (detectSentiment). Si el tono del texto arroja un resultado categóricamente NEGATIVE con una puntuación extrema (mayor a 0.95), el sistema asume hostilidad o lenguaje abusivo y restringe la publicación.

4. Tolerancia a Fallos y Mecanismo de Resiliencia (Fallback)

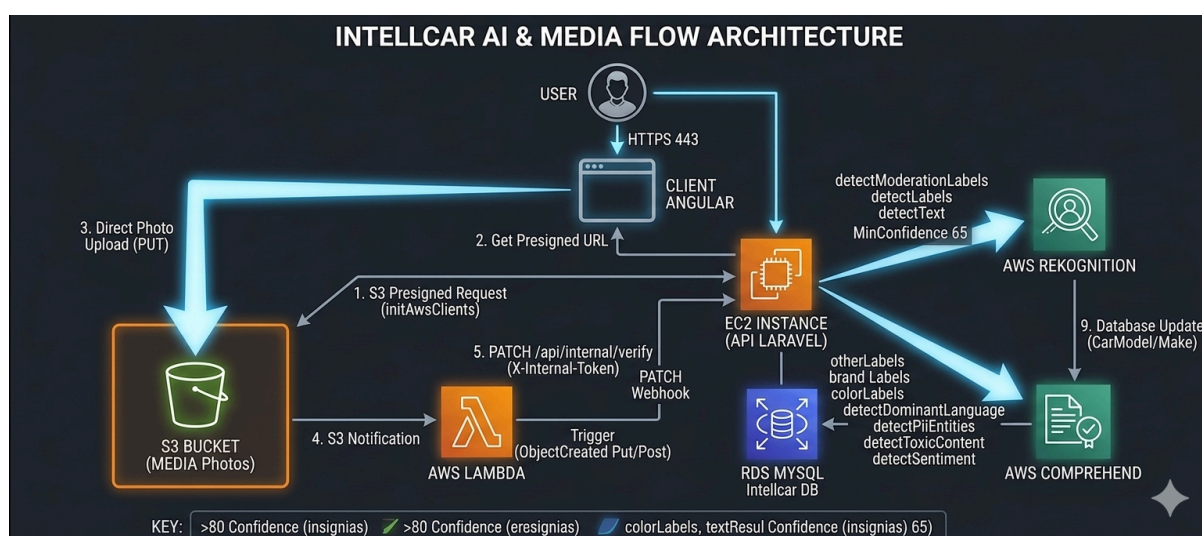
Dado que el proyecto se ejecuta bajo una infraestructura de pruebas académicas que puede verse sujeta a cuotas de consumo limitadas, restricciones de conectividad o suspensiones en la cuenta de AWS Learner Lab, ambos controladores incorporan una arquitectura de código defensivo mediante capturas de excepciones globales (try-catch de tipo Throwable):

Si cualquiera de los servicios cloud de Inteligencia Artificial (Rekognition o Comprehend) experimenta una caída o denegación de servicio, la aplicación activa de forma transparente un motor de procesamiento local de emergencia:

En el caso del texto, se ejecutan potentes expresiones regulares (preg_match) optimizadas para detectar cadenas de correo electrónico y formatos de números telefónicos nacionales e internacionales, además de aplicar un filtro de expresiones malsonantes (Badwords) predefinido por software.

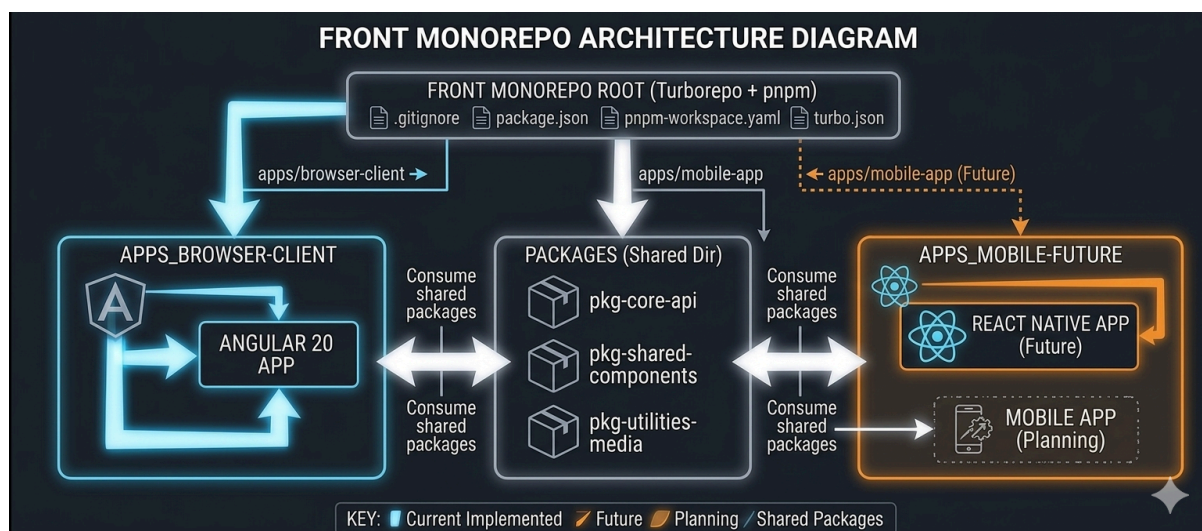
El sistema aprueba la transacción inyectando una advertencia informativa amigable en el JSON de respuesta (warning).

Este diseño garantiza la alta disponibilidad de la aplicación, demostrando un enfoque pragmático de ingeniería de software donde el sistema es capaz de degradar sus funciones de manera elegante sin interrumpir la experiencia de usuario ni detener el funcionamiento del ecosistema Intellcar.



4.2 Frontend (Turborepo)

Estructura del Monorepo (pnpm/ Turbo)



Para el desarrollo del ecosistema cliente de Intellcar, se ha descartado la estructura tradicional de repositorios independientes (polyrepos) en favor de una arquitectura de Monorepo.

1. ¿Qué es un Monorepo y por qué se aplica en Intellcar?

Un monorepo es una estrategia de gestión de software donde múltiples aplicaciones, proyectos o librerías independientes coexisten dentro de un único repositorio de Git, compartiendo configuraciones, herramientas y dependencias, pero manteniendo fronteras de desarrollo claras.

En nuestro proyecto, esta decisión responde a una estrategia de escalabilidad y reutilización de código a largo plazo:

Estructura Actual: El monorepo aloja actualmente la aplicación web principal bajo el directorio `apps/browser-client`, desarrollada con Angular 20. De forma paralela, cuenta con una carpeta `packages/` destinada a albergar librerías compartidas (tipados lógicos de TypeScript, llamadas API centralizadas, validadores de formularios y utilidades de media).

Proyección de Futuro (React Native): La razón de peso para centralizar el desarrollo en un monorepo es la futura expansión de Intellcar hacia dispositivos móviles. El proyecto está diseñado y preparado para integrar de forma nativa una aplicación en React Native (que se ubicará en `apps/mobile-app`). Gracias a esta arquitectura, la futura app móvil podrá consumir exactamente los mismos paquetes compartidos de lógica de negocio, tipados de datos de los vehículos y conectores de la API REST

que ya utiliza la app de Angular, reduciendo el tiempo de desarrollo a la mitad y evitando la duplicación de código (DRY - Don't Repeat Yourself).

2. Gestión de Dependencias: Por qué pnpm frente a npm

Para gestionar este ecosistema de manera eficiente se ha seleccionado pnpm (Performant npm) como gestor de paquetes principal, configurado a través del archivo pnpm-workspace.yaml. Su elección frente al estándar tradicional de npm se justifica bajo tres pilares fundamentales:

A. Eficiencia de Almacenamiento mediante Enlaces Duros (Hard Links)

A diferencia de npm, que copia físicamente los paquetes de node_modules en cada proyecto de forma redundante (llenando gigabytes de espacio en disco), pnpm utiliza un almacenamiento direccionable por contenido. Todos los paquetes se descargan una única vez en un directorio global de la máquina. Luego, pnpm crea enlaces duros (hard links) y simbólicos desde los proyectos hacia ese almacén global. Esto optimiza drásticamente el espacio en disco y reduce el tiempo de instalación en los flujos de Integración Continua (CI/CD).

B. Eliminación de Dependencias Fantasma (Phantom Dependencies)

npm crea un árbol de node_modules plano. Esto significa que si una librería que instalas depende de una "librería X", tú puedes importar la "librería X" en tu código aunque no la hayas declarado explícitamente en tu package.json. Esto es un riesgo técnico crítico, ya que si la primera librería actualiza o elimina su dependencia, tu aplicación se rompe. pnpm mitiga esto por completo: utiliza una estructura de enlaces estricta que solo permite a tu código acceder a las dependencias que has declarado explícitamente, garantizando la estabilidad del software.

C. Seguridad de la Cadena de Suministro (Supply Chain Security)

El ecosistema tradicional de npm ha sufrido graves brechas de seguridad y ataques de inyección de código malicioso en su registro público en los últimos años (como el secuestro de paquetes populares mediante técnicas de typosquatting o robo de credenciales de mantenedores, comprometiendo servidores de producción).

Al enfrentarnos a un proyecto inteligente conectado con servicios cloud críticos (AWS), la seguridad es prioritaria. pnpm ofrece una capa defensiva superior frente a npm:

Verificación Estricta: Almacena e indexa los paquetes basándose en su código hash criptográfico. Si un atacante lograra alterar el código de una librería en el registro público, pnpm detectaría que el hash no coincide con el almacén global y detendría la instalación inmediatamente.

Aislamiento en Monorepos: Su estructura basada en enlaces impide que dependencias maliciosas ocultas e indirectas escalen privilegios o lean ficheros de configuración de otras aplicaciones del entorno de trabajo, blindando el código del Frontend de Intellcar.

Experiencia de usuario con GSAP y Motion

Para elevar la experiencia de usuario a estándares profesionales y dotar a la interfaz de Intellcar de transiciones fluidas, orgánicas y reactivas, se han integrado dos de las soluciones de animación más potentes y optimizadas del ecosistema web actual:

1. GSAP (GreenSock Animation Platform)

GSAP es el estándar de la industria para animaciones complejas y de alto rendimiento. No es una simple librería de CSS, sino un motor de animación por JavaScript extremadamente rápido que manipula directamente las propiedades de los elementos del DOM.

¿Por qué se usa en el proyecto? Se ha seleccionado para gestionar las microinteracciones del ecosistema web, las líneas de tiempo coordinadas (Timelines) y animaciones complejas que dependen del desplazamiento del usuario (ScrollTrigger). GSAP destaca por su rendimiento inigualable, minimizando el jank (caídas de fotogramas) y garantizando que las transiciones visuales se ejecuten a 60 FPS estables, incluso en dispositivos con recursos limitados.

2. Motion.dev (Motion)

Motion (desarrollada por los creadores de Framer Motion) es una librería de animación moderna de nueva generación, diseñada específicamente para ser ligera, declarativa y aprovechar las APIs nativas del navegador, como la Web Animations API (WAAP).

¿Por qué se usa en el proyecto? Mientras que GSAP se encarga de las secuencias e interactividad compleja, Motion se utiliza para la animación de componentes reactivos y transiciones de estado básicas. Su sintaxis declarativa se integra a la perfección con arquitecturas modernas de componentes, permitiendo animar de forma limpia elementos cuando se montan o desmontan del DOM (como modales, alertas dinámicas o la carga de tarjetas de vehículos) ocupando apenas unos pocos kilobytes en el bundle final.

Beneficio Arquitectónico: Rendimiento en el Navegador

Al combinar ambas librerías, el Frontend de Intellcar delega todo el cálculo visual a la GPU (tarjeta gráfica) del dispositivo en lugar de saturar la CPU (el hilo principal de ejecución de JavaScript). Esto asegura que la interfaz se sienta instantánea, pulida y con un comportamiento premium que acompaña la identidad tecnológica del proyecto.

4.2 Infraestructura como Código con Terraform (IaC)

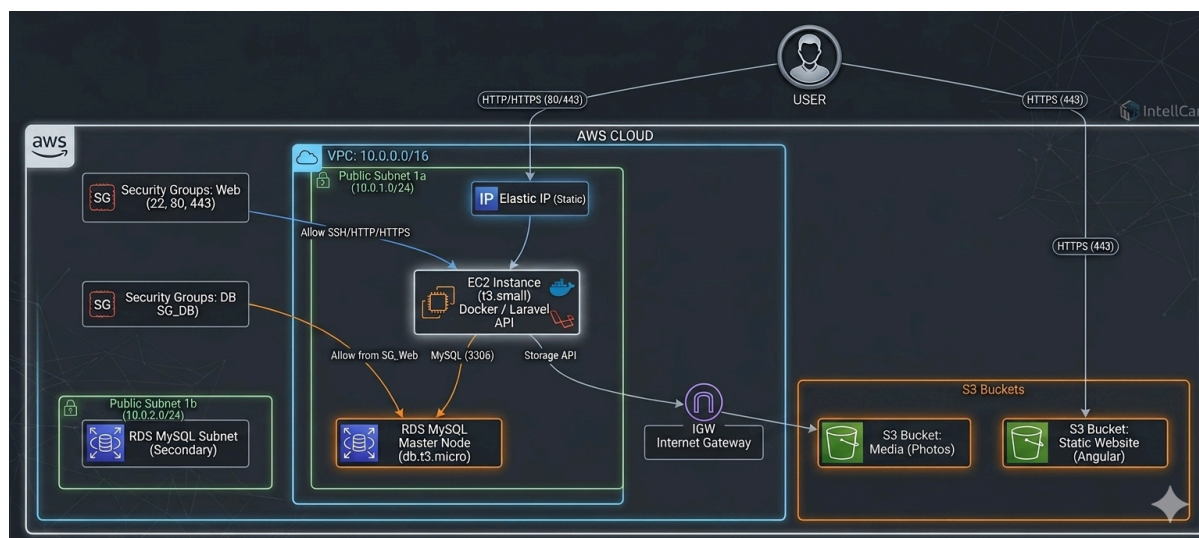
Tradicionalmente, la configuración de los servidores, las redes y las bases de datos se realizaba de forma manual a través de interfaces web o comandos directos en la consola. Este enfoque suele propiciar errores humanos y dificulta la réplica exacta del entorno.

Para este proyecto, se ha optado por un enfoque moderno basado en **Infraestructura como Código (IaC)**. Esto significa que toda la estructura de servidores y servicios que necesita nuestra aplicación web no se crea a mano, sino que se define y documenta mediante archivos de texto (código).

¿Por qué Terraform?

Porque es la solución para gestionar una infraestructura “en condiciones” por las siguientes razones estratégicas:

1. **Automatización y Control del Entorno:** Terraform permite desplegar (levantar con Terraform Apply) o eliminar (destruir con Terraform Destroy) toda la infraestructura necesaria para la aplicación mediante comandos sencillos en la terminal. Esto reduce a cero los fallos de configuración manual y agiliza los tiempos de despliegue.
2. **Reutilización y Escalabilidad:** Al estar la infraestructura escrita en código, este diseño se convierte en una plantilla. Si en el futuro se necesita replicar este mismo entorno para otro proyecto, o crear un entorno idéntico de "Pruebas" (Staging) separado del de "Producción", basta con reutilizar los mismos archivos de configuración.
3. **Independencia del Proveedor (Provider Agnostic):** Terraform no está ligado a una única empresa de servicios en la nube. Permite gestionar recursos tanto en AWS, como en Google Cloud, Azure o proveedores de servidores virtuales (como DigitalOcean o Linode) utilizando el mismo lenguaje de configuración. Esto aporta flexibilidad al proyecto si se decide cambiar de proveedor en el futuro.
4. **Historial de Cambios (Estado):** La herramienta mantiene un registro exacto (archivo de estado) de cómo está el servidor en cada momento. Si realizamos un cambio en el código, Terraform sabe exactamente qué parte del servidor real debe modificar sin necesidad de borrar y reconstruir todo desde el principio.



VPC

Para garantizar el aislamiento y la seguridad de toda la infraestructura de Intellcar, no se han utilizado las redes compartidas por defecto del proveedor de la nube. En su lugar, se ha diseñado un entorno de red virtual a medida dentro de Amazon Web Services (AWS) mediante código.

A continuación, se detalla cómo se estructura lógicamente este espacio de red y los motivos técnicos de su distribución:

1. Perímetro de Red (VPC)

Se ha creado una VPC (Virtual Private Cloud) dedicada en exclusiva al proyecto, asignándole el rango de direccionamiento 10.0.0.0/16. Esto actúa como un contenedor privado y seguro en la nube. Ningún elemento externo puede comunicarse con los recursos de Intellcar a menos que se definan explícitamente las reglas de acceso y enrutamiento, lo que elimina cualquier vulnerabilidad por exposición accidental.

2. Segmentación del Espacio (Subredes)

Para organizar los diferentes recursos de la aplicación según su función y criticidad, la VPC se ha dividido en dos subredes ubicadas en zonas geográficas separadas (Zonas de Disponibilidad us-east-1a y us-east-1b). Esta separación responde a dos necesidades fundamentales: el despliegue del servidor web y la seguridad de la base de datos.

- **Subred Pública Primaria - Public Subnet 1a (10.0.1.0/24):** Ubicada en la zona de disponibilidad 1a, está configurada para permitir la interacción con el exterior. En esta subred se aloja de forma estratégica la instancia EC2 que ejecuta el backend (API en Laravel con Docker). Al ser una subred con salida directa a internet a través de la puerta de enlace (Internet Gateway), los usuarios y el frontend pueden realizar peticiones HTTP/HTTPS al servidor sin restricciones de red.

- **Subred de Respaldo y Requisito de Base de Datos - Public Subnet 1b (10.0.2.0/24):**

Esta segunda subred se ha desplegado de forma aislada en la zona de disponibilidad 1b. Su creación responde estrictamente a una medida de seguridad, alta disponibilidad y arquitectura para nuestro motor de base de datos RDS MySQL.

AWS requiere de forma obligatoria que los servicios de bases de datos gestionados (RDS) se asocien a un grupo de subredes que abarque, como mínimo, dos zonas de disponibilidad distintas. El motivo técnico es doble:

- **Seguridad y Aislamiento:** Aunque en el esquema la subred comparta el direccionamiento base, actúa como un entorno separado para el nodo secundario o de réplica de la base de datos, asegurando que la infraestructura de datos no dependa de un único punto de fallo físico.
- **Tolerancia a Fallos (Alta Disponibilidad):** Si el centro de datos de la zona 1a sufriera una caída o un problema técnico grave, AWS podría levantar o mantener la integridad de los datos utilizando los recursos de la zona 1b, garantizando que la información del proyecto Intellcar no se pierda y el sistema sea resiliente.

EC2

El núcleo computacional donde se ejecuta la lógica de negocio y el backend de Intellcar se encuentra centralizado en un servidor virtual en la nube. En lugar de utilizar un servidor físico tradicional, se ha desplegado una instancia escalable que procesa todas las peticiones del sistema.

A continuación, se describen las características técnicas de este servidor y la tecnología utilizada para desplegar la aplicación de forma eficiente:

1. Dimensionamiento del Servidor (Amazon EC2)

Para el alojamiento de la API se ha seleccionado el servicio Amazon EC2 (Elastic Compute Cloud), optando específicamente por un tipo de instancia t3.small.

La elección de este hardware virtualizado responde a un equilibrio óptimo entre coste y rendimiento para la fase actual del proyecto:

Capacidad de Cómputo: Cuenta con 2 vCPUs (procesadores virtuales) y 2 GiB de memoria RAM. Esta configuración es más que suficiente para gestionar con fluidez el servidor web, el entorno de ejecución de PHP y las peticiones concurrentes de los usuarios a la API sin sufrir degradación de rendimiento.

Arquitectura de Créditos de CPU: Al pertenecer a la familia t3, la instancia acumula créditos cuando el servidor está en reposo y los consume cuando hay picos de tráfico (por ejemplo, durante la simulación de uso por parte de los profesores o múltiples peticiones simultáneas), garantizando una alta disponibilidad económica.

2. Contenedores de Aplicación (Docker)

En lugar de instalar el servidor web, PHP y las dependencias directamente sobre el sistema operativo de la máquina virtual (lo que se conoce como una instalación "nativa"), se ha decidido aislar la API de Intellcar utilizando Docker.

La API, desarrollada en el framework Laravel, "vive" y se ejecuta por completo dentro de un contenedor Docker de forma independiente. Esta decisión arquitectónica aporta los siguientes beneficios profesionales al proyecto:

Portabilidad Absoluta: Docker empaqueta el código de la API junto con la versión exacta de PHP, las extensiones necesarias y las configuraciones del servidor web en una sola "imagen". Esto garantiza el principio de "funciona en mi ordenador y funciona en el servidor", eliminando por completo los clásicos errores de despliegue por incompatibilidad de versiones entre el entorno de desarrollo local y el entorno de producción en AWS.

Facilidad de Despliegue y Mantenimiento: Gracias a esto, actualizar la aplicación en producción es tan sencillo como descargar la nueva versión de la imagen del contenedor y reiniciarla. El proceso toma apenas unos segundos, minimizando el tiempo de inactividad de la web de Intellcar.

Aislamiento de Entornos: Al ejecutarse dentro de un entorno estanco, la aplicación no interfiere con los archivos del sistema operativo base de la EC2. Si en el futuro se quisieran añadir más microservicios o herramientas al servidor, cada una iría en su propio contenedor sin riesgo de que las dependencias de una rompan la otra.

RDS

El almacenamiento y la integridad de la información de Intellcar se gestiona a través de una base de datos relacional de nivel empresarial. En lugar de instalar un motor de base de datos dentro del propio disco del servidor EC2 (lo cual comprometería el rendimiento y la seguridad), se ha optado por Amazon RDS (Relational Database Service).

RDS es un servicio completamente gestionado por AWS que automatiza tareas complejas como los parches de seguridad, las copias de respaldo y el mantenimiento del hardware, permitiendo que la aplicación disponga de un entorno de datos altamente disponible y optimizado.

A continuación, se justifican las decisiones de configuración e implementación aplicadas mediante Terraform:

1. Motor de Base de Datos y Dimensionamiento

Se ha desplegado una instancia de MySQL en su versión 8.0, un estándar de la industria ampliamente compatible con el framework Laravel del backend.

Tipo de Instancia (db.t3.micro): Al igual que con el servidor web, se ha seleccionado un hardware optimizado para entornos de desarrollo y pruebas. Cuenta con recursos suficientes para procesar las consultas, transacciones y relaciones de tablas de la aplicación sin incurrir en costes elevados.

Almacenamiento Continuo: Se han asignado 20 GiB de disco de estado sólido (gp2). Este espacio ofrece una velocidad de lectura/escritura (IOPS) constante y balanceada, asegurando que la API obtenga las respuestas del servidor de datos en milisegundos.

2. Aislamiento y Grupo de Subredes (db_subnet_group)

Como medida de diseño de red orientada a la alta disponibilidad y seguridad, se ha creado un grupo de subredes específico para la base de datos. Este grupo asocia el servicio RDS de forma simultánea a nuestras dos subredes de la VPC (public_subnet y public_subnet_2), distribuidas en dos Zonas de Disponibilidad geográficamente aisladas.

Esto garantiza que la infraestructura de datos cumpla con los estándares de AWS para la tolerancia a fallos, protegiendo la información ante cualquier caída imprevista en uno de los centros de datos del proveedor.

3. Seguridad Perimetral y Regla de Interconexión Exclusiva

El aspecto más crítico en la protección de los datos de Intellcar radica en la configuración de su cortafuegos. El Grupo de Seguridad de la base de datos (intellcar-db-sg) aplica una restricción absoluta:

Puerto Estándar 3306 (MySQL): Solo se permite el tráfico entrante a través de este puerto si la petición se origina de forma explícita desde el Grupo de Seguridad del servidor web (web_sg).

Este enfoque técnico crea un blindaje en la arquitectura: la base de datos es completamente invisible e inaccesible desde el internet público. Ningún agente externo puede realizar ataques de fuerza bruta o intentar conexiones directas; la única vía de comunicación pasa obligatoriamente por la mediación y lógica de seguridad de nuestra API en la EC2.

4. Inyección Segura de Credenciales

En línea con las buenas prácticas de seguridad en el desarrollo de software (DevOps), el archivo de configuración evita almacenar las credenciales de acceso (como la contraseña del administrador) en texto plano. Para ello, se emplea el uso de variables parametrizadas (var.db_password).

Este mecanismo permite inyectar la contraseña en el momento exacto del despliegue desde un entorno seguro y privado, garantizando que los datos sensibles nunca queden expuestos en el código fuente ni en los repositorios del proyecto.

S3

Para el almacenamiento de archivos multimedia y el despliegue de la interfaz de usuario de Intellcar, se ha optado por Amazon S3 (Simple Storage Service). S3 es un servicio de almacenamiento basado en objetos que destaca por su alta disponibilidad, seguridad y bajo coste, evitando sobrecargar el disco duro principal del servidor EC2.

Siguiendo una arquitectura desacoplada, se han configurado dos contenedores de almacenamiento (Buckets) totalmente independientes mediante código, cada uno con una finalidad específica:

1. Bucket Multimedia: Almacenamiento de Imágenes (intellcar-media)

Este espacio está dedicado exclusivamente a guardar los recursos multimedia de la plataforma, como las fotografías de los anuncios de vehículos, las publicaciones del foro (posts) y las imágenes de los garajes de los usuarios.

Para su correcto funcionamiento con la aplicación, se ha aplicado la siguiente configuración técnica:

Políticas de Acceso Público de Lectura: Por defecto, los buckets de S3 son estrictamente privados. Se ha configurado una política de acceso explícita (s3:GetObject) que permite que cualquier usuario de internet pueda visualizar y cargar las imágenes en su navegador al navegar por la app.

Configuración CORS (Cross-Origin Resource Sharing): Al estar las imágenes alojadas en un dominio de AWS diferente al de la aplicación de Angular o la API de Laravel, los navegadores bloquearían la carga de archivos por seguridad. Para solucionarlo, se han definido reglas CORS que autorizan los métodos GET, POST y PUT, permitiendo que la web interactúe con el almacenamiento de forma segura y sin conflictos de origen.

2. Bucket Web: Alojamiento del Frontend en Angular (intellcar-web)

En lugar de alojar la interfaz web dentro del servidor EC2 junto a la API, se ha utilizado la característica de S3 para actuar como un servidor de páginas web estáticas (Static Website Hosting). Este bucket almacena los archivos compilados en producción (HTML, CSS y JavaScript) de nuestro frontend desarrollado en Angular.

Su despliegue cuenta con una lógica de enrutamiento adaptada:

Documento de Índice (index.html): Se establece como el punto de entrada principal para cualquier usuario que acceda a la URL de la web.

Gestión de Errores para Aplicaciones SPA: Dado que Angular funciona como una Single Page Application (SPA) y gestiona las rutas internamente desde el navegador, si un usuario recarga la página en una ruta secundaria (por ejemplo, /garaje), el servidor S3 de AWS no encontrará físicamente esa carpeta y lanzaría un error 404. Para solucionarlo, se ha configurado estratégicamente que el documento de error

redirija también al `index.html`. De esta forma, Angular intercepta la petición y renderiza la vista correcta sin romper la experiencia del usuario.

Política de Lectura de Código: Al igual que el bucket de imágenes, cuenta con los permisos necesarios desbloqueados para que los navegadores web de los clientes puedan descargar y ejecutar de forma pública los scripts y estilos necesarios para renderizar la aplicación de Intellcar.

Elastic IP

Por defecto, las instancias de servidores en la nube (EC2) reciben una dirección IP pública dinámica. Esto significa que si el servidor se reinicia por mantenimiento o actualizaciones, la dirección IP cambia automáticamente, lo que rompería la conexión con cualquier dominio externo.

Para solucionar este problema, se ha reservado e implementado una IP Elástica (Elastic IP) en AWS. Esta herramienta nos proporciona una dirección IP pública estática y permanente asignada a nuestra instancia de backend. Su uso es un requisito indispensable en el proyecto por los siguientes motivos:

Enlace con el Dominio: Permite asociar de forma fija nuestro nombre de dominio público (`www.intellcar.duckdns.org`) a través del servicio DuckDNS) hacia el servidor sin riesgo de que la web quede inaccesible.

Seguridad y Cifrado (Certbot/SSL): Al contar con una IP fija y un dominio estable, se ha podido implementar con éxito un certificado de seguridad criptográfico mediante Certbot (Let's Encrypt). Esto asegura que todo el tráfico web se realice bajo el protocolo seguro HTTPS (puerto 443), garantizando la privacidad de los datos de los usuarios y cumpliendo con las directrices de seguridad de las aplicaciones web modernas.

Lambda

Con el fin de mantener un desacoplamiento eficiente entre el almacenamiento multimedia y la lógica de negocio, se ha implementado un patrón de Arquitectura Orientada a Eventos. En lugar de obligar al servidor principal (EC2) a monitorizar constantemente el almacenamiento o bloquear el hilo de ejecución esperando a que se procesen las imágenes, el sistema delega esta tarea en un servicio informático sin servidor (Serverless): AWS Lambda.

A continuación, se detalla la configuración de la función, el desencadenador (trigger) y la lógica de interconexión con el backend de Intellcar:

1. Definición del Servicio Serverless (AWS Lambda)

La función se ha desplegado utilizando el entorno de ejecución Python 3.11. Al tratarse de una solución Serverless, la función permanece inactiva y no consume recursos (ni costes) de computación hasta el momento exacto en el que es requerida.

Optimización de Recursos: Se le ha asignado un tamaño de memoria balanceado de 256 MB y un tiempo de espera máximo (timeout) de 30 segundos, margen óptimo para garantizar que la comunicación HTTP externa finalice correctamente.

2. Mecanismo de Desencadenamiento (S3 Bucket Notification)

Para automatizar el proceso, se ha configurado un intermediario de comunicación (Trigger Permission) que permite al servicio Amazon S3 invocar directamente la función Lambda, la cuál es un script con funciones desarrollado en Python que analiza de forma asíncrona los metadatos suministrados por el evento de S3 y coordina la seguridad de la plataforma mediante el siguiente flujo estructurado:

Extracción de Parámetros: El script decodifica e interpreta el payload del evento para identificar con precisión el nombre del bucket y la ruta exacta (object key) de la nueva imagen que se acaba de subir.

Consumo de Variables de Entorno: Para evitar malas prácticas de acoplamiento rígido, la URL del backend (INTERNAL_API_URL) y el token de seguridad se inyectan en tiempo de ejecución a través de variables de entorno protegidas.

Petición Segura (PATCH): La función empaqueta los datos en formato JSON y realiza una petición HTTP asíncrona utilizando el método PATCH hacia un endpoint interno del backend de Laravel, incluyendo una cabecera de autenticación (X-Internal-Token) para garantizar que la petición proviene exclusivamente de nuestra infraestructura interna.

Activación del Análisis de IA (AWS Rekognition): Una vez que Laravel recibe este aviso de la Lambda confirmando que la imagen está físicamente en S3, el backend invoca el servicio de Inteligencia Artificial AWS Rekognition. Este servicio analiza el contenido de la fotografía (por ejemplo, para verificar que

corresponde a un coche real en los anuncios o posts, o para detectar contenido inapropiado) antes de dar por válida la publicación en Intellcar.

La ejecución se desencadena de forma exclusiva bajo las siguientes condiciones estratégicas:

Eventos de Creación (ObjectCreated): La Lambda solo se despierta cuando se detecta una subida exitosa de archivos mediante los métodos Put o Post en el bucket multimedia (intellcar-media).

Filtrado por Extensión: Con el objetivo de optimizar el rendimiento y no desperdiciar ciclos de cómputo, se han establecido filtros de sufijo para que la infraestructura reaccione únicamente ante imágenes con formato estándar: .jpg, .jpeg y .png. Cualquier otro tipo de archivo residual es ignorado por el disparador.

3. Lógica del Script y Comunicación Segura (Webhook Interno)

El código desarrollado en la función analiza de forma asíncrona los metadatos suministrados por el evento de S3 y coordina la notificación hacia la aplicación mediante el siguiente flujo estructurado:

Extracción de Parámetros: El script decodifica e interpreta el payload del evento para identificar con precisión el nombre del bucket y la ruta exacta (object key) de la nueva imagen almacenada.

Consumo de Variables de Entorno: Para evitar malas prácticas de acoplamiento rígido, la URL del backend (INTERNAL_API_URL) y el token de seguridad se inyectan en tiempo de ejecución a través de variables de entorno protegidas de AWS.

Petición Segura (PATCH): La función empaqueta los datos en formato JSON y realiza una petición HTTP asíncrona utilizando el método PATCH hacia un endpoint interno y protegido de nuestra API de Laravel.

Validación por Token (X-Internal-Token): Para blindar el endpoint del backend y garantizar que nadie ajeno a la infraestructura pueda simular subidas de imágenes, la petición incluye una cabecera de seguridad personalizada. Laravel intercepta este token y solo procesa la información si coincide con la clave interna compartida, asegurando la integridad lógica de la aplicación.

Security Groups

A) Grupo de Seguridad del Servidor Web (**Security Group: Web**)

Este grupo está asociado directamente a la instancia EC2 que ejecuta nuestra API y el contenedor Docker. Está diseñado para gestionar las conexiones externas de los usuarios y administradores, permitiendo la entrada únicamente a través de tres puertos específicos:

- **Puerto 22 (SSH):** Permitido exclusivamente para tareas de administración del servidor, despliegues y mantenimiento mediante la terminal.
- **Puerto 80 (HTTP):** Abierto para permitir las peticiones web iniciales de los clientes antes de ser redirigidas a la capa segura.
- **Puerto 443 (HTTPS):** El canal principal de la aplicación. Permite la entrada de tráfico cifrado para que el frontend (alojado en el Bucket S3) y los usuarios puedan consumir los servicios de la API de Intelcar de forma segura.

B) Grupo de Seguridad de la Base de Datos (**Security Group: DB**)

El almacenamiento de la información es el activo más crítico del proyecto y, por lo tanto, no debe tener ninguna exposición a internet. Este grupo de seguridad protege nuestro servicio RDS MySQL aplicando una regla de aislamiento estricta:

Puerto 3306 (MySQL): Se ha configurado una regla de entrada cerrada al mundo exterior. La base de datos solo acepta peticiones que provengan del Grupo de Seguridad Web (EC2).

De este modo, aunque un atacante intentase conectarse directamente a la base de datos desde internet, el cortafuegos bloqueará la conexión de inmediato. La única forma de interactuar con los datos de Intelcar es que la petición sea tramitada y validada previamente por nuestra propia aplicación en la EC2, blindando por completo el almacenamiento del sistema.

5 — Seguridad, Calidad y Despliegue

Moderación automática por IA

El ecosistema de Intellcar no es solo un agregador de datos técnicos de automoción; es una plataforma social y un mercado donde interactúan personas. En cualquier entorno digital abierto, la libertad de publicación sin control es un vector de riesgo crítico.

Para mitigar esto, se ha implementado una arquitectura de seguridad proactiva basada en IA (AWS Rekognition y AWS Comprehend). A continuación, se analiza este enfoque desde la filosofía de la seguridad integral, comparándolo con los modelos tradicionales de revisión manual o la total ausencia de moderación.

1. Los Tres Escenarios de Gestión de Contenido: Análisis Comparativo

Para justificar la necesidad de este módulo, debemos analizar el impacto en la seguridad de la plataforma y del usuario según el enfoque adoptado:

Criterio de Seguridad	Escenario A: Sin Moderación (Anarquía)	Escenario B: Revisión Manual (Reactiva/Lenta)	Escenario C: Moderación con IA en Intellcar (Proactiva)
Protección Legal (Responsabilidad)	Nula. La plataforma es cómplice legal de contenidos delictivos alojados en sus servidores.	Baja-Media. El contenido delictivo es visible públicamente hasta que un administrador lo revisa y lo borra.	Máxima. El contenido malicioso o ilegal se intercepta <i>en caliente</i> antes de llegar a ser público.
Privacidad del Usuario (PII)	Inexistente. Filtraciones constantes de teléfonos, emails y estafas en abierto.	Tardía. Los datos privados quedan expuestos durante horas o días a merced de <i>scrapers</i> (bots de extracción).	Inmediata. El texto se sanitiza en milisegundos, impidiendo que el dato privado se exponga.

Experiencia en la Comunidad	Degradación inmediata por <i>spam</i> , contenido adulto, estafas y toxicidad.	Frustración del usuario (los anuncios tardan horas en ser aprobados por el administrador).	Fluida y segura. Validación instantánea en segundo plano sin fricción.
-----------------------------	--	--	--

2. Por qué la Moderación Manual es un Enemigo de la Seguridad Real

Dejar la seguridad en manos de un panel de administración donde un humano valida los posts uno a uno presenta tres fallas filosóficas y operativas:

- La Ventana de Vulnerabilidad Temporal: Si un usuario sube una estafa o contenido ofensivo a las 3:00 AM y el administrador no entra hasta las 9:00 AM, la plataforma ha sido insegura durante 6 horas. En ese tiempo, un bot puede rastrear el teléfono de la víctima o un menor puede ver una imagen inapropiada. La seguridad manual siempre llega tarde.
- El Sesgo Humano y la Fatiga: Un administrador humano sufre fatiga cognitiva. Tras revisar 200 anuncios, su capacidad para detectar una matrícula oculta, un patrón de texto fraudulento o un insulto sutil disminuye drásticamente. La IA mantiene el mismo umbral estricto de confianza (\$65\%\$ o \$80\%\$) de forma constante las 24 horas del día.
- El Factor de la Privacidad del Moderador: Exponer a los administradores a revisar manualmente contenido potencialmente explícito, violento o ilegal genera problemas éticos y de salud mental, un riesgo laboral real en las grandes tecnológicas que *Intellcar* erradica automatizando el primer filtro.

3. La IA como Escudo de Seguridad Bidireccional (Plataforma y Usuario)

La implementación en los controladores **RekoControl** y **ComprehendControl** actúa como un escudo de seguridad bidireccional:

A. Protección para el Usuario (Seguridad Humana)

- Privacidad por Diseño (Anti-PII): Los usuarios, a menudo por descuido, publican su teléfono o email en la descripción de un coche. Esto los convierte en blanco de ataques de *phishing*, acoso o estafas fuera de la plataforma. AWS Comprehend actúa de guardián impidiendo que el usuario se ponga en peligro a sí mismo.
- Entorno Psicológicamente Seguro (Anti-Toxicidad): El foro y los comentarios están blindados contra el discurso de odio y la hostilidad extrema. Al cortar la toxicidad de raíz, se fomenta una comunidad sana y comercialmente atractiva.

B. Protección para la Plataforma (Seguridad de Negocio y Legal)

- Blindaje Reputacional y de Infraestructura: AWS Rekognition (**detectModerationLabels**) evita que los servidores de *Intellcar* y sus buckets de S3 almacenen de forma ilegal imágenes explícitas, violentas o con derechos de autor

protegidos. Esto protege a la empresa de demandas legales y de ser catalogada como "sitio inseguro" por los motores de búsqueda.

-
- Autenticidad del Modelo de Negocio: Al exigir que la IA detecte obligatoriamente etiquetas de vehículos (**REQUIRED_VEHICLE_LABELS**), evitamos el *spam* de usuarios que intentan vender productos ajenos al motor o romper la temática de la app, manteniendo la base de datos limpia y optimizada.

En *Intellcar*, la Inteligencia Artificial no se utiliza como un añadido estético para inflar el valor tecnológico del proyecto; se utiliza como una política automatizada de gobernanza. Compendia el equilibrio perfecto: ofrece al usuario la gratificación instantánea de ver su anuncio publicado al momento (sin esperas de un administrador), pero con la total garantía de que la plataforma ha auditado el 100\% de los bits de esa imagen y ese texto para certificar que el ecosistema sigue siendo seguro, limpio y confiable.

Dockerización en entornos locales en desarrollo y en AWS en producción.

Para garantizar la portabilidad, la homogeneidad de los entornos y un ciclo de vida de desarrollo ágil, el ecosistema de Intellcar se ha diseñado desde su origen utilizando Docker y Docker Compose.

A continuación, se detalla la justificación técnica de la containerización, el análisis de los archivos de configuración empleados y el impacto crítico (los problemas que se han evitado) tanto en los entornos locales de desarrollo como en la instancia de producción en la nube (AWS EC2).

Justificación Técnica: ¿Por qué containerizar con Docker?

La adopción de Docker en Intellcar responde a la necesidad de aislar la aplicación de la máquina física donde se ejecuta. Al empaquetar el servidor web Apache, el intérprete de PHP 8.3 con sus extensiones y las herramientas de dependencias (Composer) en imágenes inmutables, se consigue el principio de "Desarrolla una vez, ejecuta en cualquier lugar".

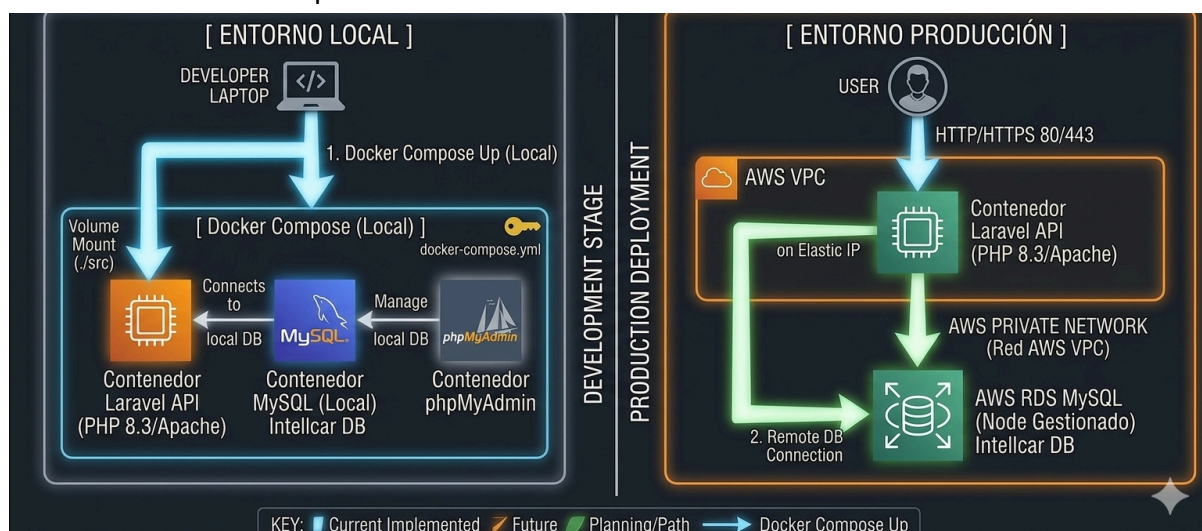
El problema de los permisos en sistemas híbridos (Windows/WSL)

Un desafío técnico real durante el desarrollo local fue la diferencia en la gestión de sistemas de archivos entre Windows y entornos basados en Linux (como WSL o servidores de producción).

En el Dockerfile, se implementó una directiva correctiva avanzada para solucionar el conflicto de permisos de escritura sobre las carpetas de caché y logs de Laravel (storage y bootstrap/cache):

RUN usermod -u 1000 www-data && groupmod -g 1000 www-data

Por defecto, el usuario www-data de Apache dentro del contenedor corre bajo el UID 33. Al mapearlo explícitamente al UID 1000 (el ID de usuario estándar en sistemas GNU/Linux y WSL), se permite que el desarrollador edite el código en tiempo real desde el host sin que el contenedor pierda permisos de ejecución sobre el sistema de archivos compartido a través de los volúmenes de Docker.



Entorno de Desarrollo (Local): Incluye un stack completo para que la aplicación sea 100% autónoma. Levanta el contenedor de la API (Laravel), un contenedor local con el motor de base de datos MySQL, y una interfaz gráfica phpMyAdmin para facilitar la depuración, pruebas de siembra de datos (seeders) y migraciones sin necesidad de instalar software en la máquina del desarrollador.

Entorno de Producción (docker-compose.prod.yml): En producción (instancia AWS EC2), la arquitectura se vuelve minimalista y de alta disponibilidad. El archivo de Compose solo levanta el contenedor de la API de Laravel (Apache + PHP). No hay rastro de MySQL local ni de phpMyAdmin. Esto reduce drásticamente la superficie de ataque del servidor, optimiza la memoria RAM de la máquina EC2 y delega la persistencia de datos de forma segura en AWS RDS (Relational Database Service) mediante la red privada de la VPC.

Matriz de Impacto: Problemas evitados gracias a Docker

Para comprender el valor real de esta implementación, es necesario analizar el escenario catastrófico al que se habría enfrentado el proyecto si se hubiera desplegado mediante un método tradicional (instalación manual sobre el sistema operativo):

Escenario	Problemas SIN Docker (Instalación Tradicional)	Solución CON Docker en Intellcar
Local (Desarrollo)	El síndrome del "En mi máquina funciona": Si el desarrollador tiene PHP 8.2 en su ordenador y el servidor usa PHP 8.3, se producen errores sintácticos invisibles en local. Además, instalar extensiones pesadas de PHP (pdo_mysql, gd, mbstring) difiere totalmente entre Windows, macOS y Linux, frustrando el inicio del proyecto.	Entorno idéntico e instantáneo: Ejecutando un único comando (<code>docker compose up</code>), cualquier máquina del mundo recrea exactamente el mismo entorno con PHP 8.3-Apache y sus extensiones nativas en cuestión de segundos.

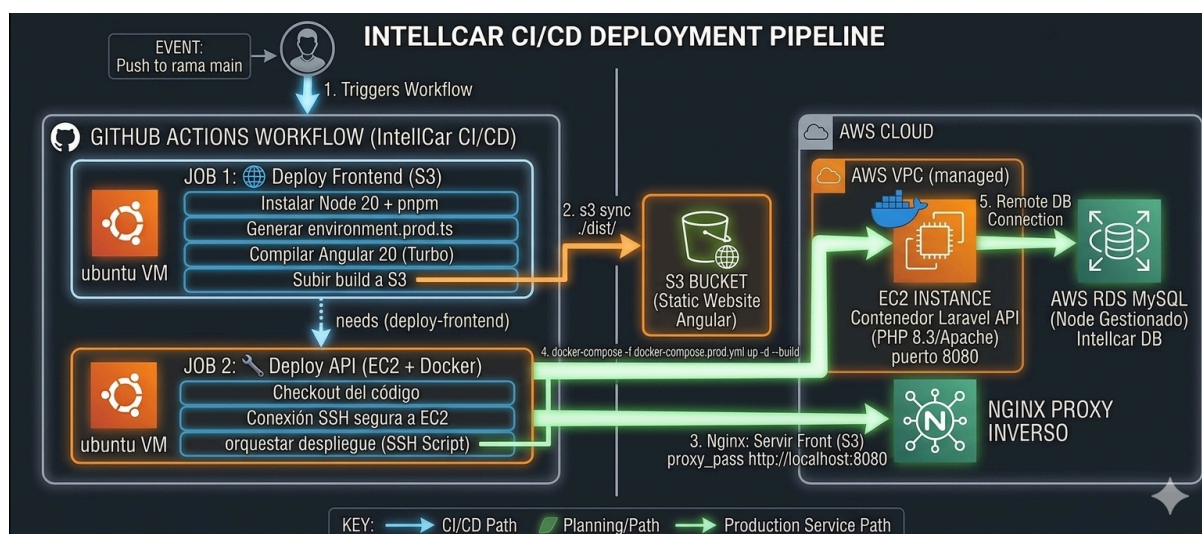
Producción (Despliegue)	Saturación y falta de aislamiento: Instalar Apache y MySQL directamente en la máquina EC2 compite de forma agresiva por la memoria RAM (\$t3.small\$). Si la base de datos local sufre un pico de consultas, puede tumbar el servidor web por completo. Además, actualizar el sistema operativo del servidor podría romper las librerías compartidas de PHP, deteniendo el servicio web de la plataforma.	Aislamiento total y desacoplamiento: El contenedor de producción es una caja estanca e inmutable. Al delegar la base de datos a AWS RDS, la instancia EC2 solo consume recursos procesando peticiones HTTP. Si el contenedor falla, la directiva <code>restart: unless-stopped</code> levanta el servicio web automáticamente en milisegundos.
Limpieza y Mantenimiento	Contaminación del Sistema Operativo: Instalar bases de datos, servidores web y dependencias globales deja residuos permanentes, variables de entorno conflictivas y puertos bloqueados en la máquina del desarrollador.	Eliminación sin rastro: El ciclo de vida de los contenedores es efímero. Si el proyecto se detiene, basta con destruir los contenedores para recuperar la máquina anfitriona completamente limpia y sin configuraciones residuales.

La containerización de Intellcar demuestra que el proyecto ha sido concebido bajo buenas prácticas de ingeniería de software y metodologías DevOps. El uso de un Dockerfile optimizado y la separación de entornos mediante múltiples archivos docker-compose garantiza que la transición del código desde la máquina de desarrollo local hacia la infraestructura de producción en AWS sea un proceso seguro, predecible, automatizado y libre de errores humanos.

Despliegue continuo (Deploy.yml con GitHub Actions)

Para automatizar el ciclo de vida de Intellcar, mitigar el error humano en los despliegues manuales y garantizar que el servidor siempre cuente con la última versión estable del código, se ha diseñado un pipeline de CI/CD mediante GitHub Actions. El flujo de trabajo está programado para dispararse de forma automática ante cualquier evento de push en la rama principal (main).

El flujo de trabajo (workflow) está dividido en dos trabajos o jobs secuenciales y acoplados: Deploy Frontend (S3) y Deploy API (EC2 + Docker). Su ejecución garantiza que la API solo se actualice si la compilación del cliente web ha sido completamente exitosa.



- Job 1: Compilación y Distribución del Frontend (deploy-frontend)

Este primer bloque se ejecuta en un entorno aislado basado en una máquina virtual de Ubuntu limpia (ubuntu-latest). Su objetivo es generar los recursos estáticos del cliente y subirlos a la nube:

Configuración del Entorno de Construcción: El pipeline realiza un checkout del repositorio, instancia Node.js 20 e instala el gestor de paquetes pnpm v9.

Instalación Determinista: Ejecuta `pnpm install --frozen-lockfile`. Esta bandera es crucial en entornos de CI/CD: obliga al gestor a respetar las versiones exactas del archivo de bloqueo, impidiendo que dependencias de terceros se actualicen en caliente y alteren la integridad de la compilación.

Inyección Dinámica del Entorno: Antes de compilar, el script genera el archivo `environment.prod.ts` en caliente:

```
echo "export const environment = { production: true, apiUrl: window.location.origin + '/api' };" > src/environments/environment.prod.ts
```

Esto permite desacoplar URLs estáticas del código; la aplicación Angular asumirá dinámicamente que la API vive bajo el mismo dominio de origen, pero apuntando al endpoint /api.

Compilación a Producción: Se ejecuta el build de Angular a través del monorepo (pnpm run build --no-cache) generando el paquete optimizado de distribución en la carpeta dist.

Sincronización Inmutable con AWS S3: Mediante la acción oficial de AWS, el pipeline se autentica con las credenciales cifradas en los secretos de GitHub (AWS_ACCESS_KEY_ID, AWS_SECRET_ACCESS_KEY y AWS_SESSION_TOKEN). Finalmente, ejecuta el comando `aws s3 sync ... --delete`, que clona de forma exacta la carpeta compilada de Angular dentro del bucket S3: Static Website, eliminando cualquier archivo residual antiguo.

- **Job 2: Despliegue de la API y Configuración del Servidor (deploy-api)**

Este trabajo requiere obligatoriamente (`needs: deploy-frontend`) que el cliente web se haya desplegado correctamente. El flujo utiliza una conexión cifrada SSH (`appleboy/ssh-action`) para tomar el control de la instancia AWS EC2 y orquestar las siguientes fases operativas:

A. Limpieza de Infraestructura y Despliegue del Código

El script se desplaza al directorio del proyecto en la máquina virtual, detiene de forma segura los contenedores que se encuentren en ejecución (`docker-compose -f docker-compose.prod.yml down`) para liberar la memoria RAM y los volúmenes del sistema, elimina los datos antiguos y realiza un clonado limpio (`git clone -b main`) desde GitHub. Esto asegura un entorno puro, libre de código huérfano.

B. Generación Segura del Entorno Estricto (.env)

En lugar de subir el archivo sensible de configuración al control de versiones (lo que supondría una vulnerabilidad grave), el pipeline reconstruye el fichero `src/.env` de producción en tiempo real dentro del servidor EC2. Esto se logra inyectando las credenciales secretas de la base de datos de producción (AWS RDS), los tokens de sesión de AWS para el uso de Rekognition / Comprehend, y la clave criptográfica de cifrado de Laravel (`APP_KEY`).

C. Orquestación del Contenedor Laravel y Gestión de Dependencias

- Se sincroniza la hora del servidor con los servidores de Ubuntu (`ntpdate`) para evitar fallos de firma criptográfica y desfases de tiempo (`clock skew`) al invocar las APIs de AWS.
- Se levanta el entorno de producción aislado mediante el comando `docker-compose -f docker-compose.prod.yml up -d --build`. Al no disponer de servicios de base de datos locales, el contenedor de Laravel arranca de forma inmediata.
- Se instalan las dependencias de producción de PHP ejecutando `composer update --no-dev --optimize-autoloader`. El modificador `--optimize-autoloader` convierte el mapa de clases asociativas de Composer en un archivo plano muy rápido, acelerando drásticamente los tiempos de respuesta de la API.

- Se ajustan los permisos del sistema de archivos asignando la propiedad al usuario web del servidor (`chown -R www-data:www-data`) y otorgando permisos de escritura (`775`) a las carpetas dinámicas `storage` y `bootstrap/cache`.

D. Preparación de la Persistencia y Caché de Rendimiento

Para dejar el sistema operativo en un estado óptimo para la producción, se ejecutan de forma secuencial herramientas nativas del framework a través del CLI de *Artisan*:

1. `migrate:fresh --seed --force`: Reconstruye el esquema de tablas directamente en AWS RDS, ejecuta los semilladores (*seeders*) para poblar los datos iniciales obligatorios del sistema, y la bandera `--force` anula la confirmación interactiva humana necesaria en entornos productivos.
2. `l5-swagger:generate`: Compila y genera el archivo JSON de la documentación interactiva de la API para que esté disponible de inmediato en la ruta `/api/documentation`.
3. Optimización de Caché (`config:cache`, `route:cache`, `view:cache`): Aplana todos los archivos de configuración, enrutamiento y plantillas en arrays estáticos de PHP. Gracias a esto, Laravel no tiene que leer decenas de ficheros del disco duro en cada petición HTTP, reduciendo el tiempo de ejecución en más de un 70%.

Configuración del Servidor Web Nginx como Proxy Inverso

El flujo finaliza configurando **Nginx** directamente en la máquina anfitriona EC2 para que actúe como la pasarela de entrada y el cerebro de enrutamiento de *Intellcar*. El pipeline escribe la configuración en `/etc/nginx/sites-available/intellcar` estructurando el tráfico bajo las siguientes reglas de enrutamiento:

- **Sincronización Local del Frontend**: El servidor descarga de forma local el contenido estático de Angular que el *Job 1* subió previamente al bucket de S3 mediante el comando `aws s3 sync`. Nginx sirve estos archivos (`index.html`, paquetes de JavaScript y CSS) de forma ultra instantánea directamente desde el disco de la máquina bajo la ruta raíz `/`.
- **Redirección de la API (Proxy Inverso)**: Cuando llega una petición dirigida a `/api`, Nginx intercepta el tráfico y lo redirige internamente al puerto `8080` (`proxy_pass http://localhost:8080`), que es donde está escuchando el contenedor Docker de Laravel. Nginx preserva las cabeceras originales del cliente (`X-Real-IP`, `X-Forwarded-For`, `X-Forwarded-Proto`), asegurando que Laravel conozca la dirección IP real del usuario para auditorías de seguridad.
- **Persistencia en Rutas Administrativas**: Las peticiones dirigidas al panel administrativo clásico (`/admin`) o a la documentación técnica (`/docs`) se desvían de igual forma hacia el contenedor de Docker para mantener el correcto funcionamiento de las sesiones basadas en servidor.

Este flujo de CI/CD representa una solución robusta y alineada con las prácticas modernas de desarrollo de software. Al segmentar la infraestructura —dejando que Nginx sirva los archivos estáticos de Angular descargados de S3 y delegando las peticiones pesadas al contenedor aislado de Laravel conectado a AWS RDS— se logra una arquitectura con alta tolerancia a fallos, eficiente en recursos y con un proceso de despliegue totalmente automatizado a golpe de git push.

Agregación del Dominio Duck DNS

El acceso a la plataforma de producción de Intellcar a través de una dirección IP pública cruda (<http://54.XX.XX.XX>) resulta inviable para el usuario final y bloquea la posibilidad de establecer conexiones seguras. Al tratarse de un entorno de proyecto y con el fin de evitar costes adicionales en registradores de dominios comerciales, se ha optado por implementar Duck DNS como proveedor de DNS dinámico (DDNS) gratuito.

El proceso de integración y resolución se estructuró en tres pasos clave:

1. Reserva del Subdominio y Token de Autenticación

Se registró el subdominio único `intellcar.duckdns.org` en la plataforma de Duck DNS. El servicio provee un token criptográfico único de seguridad que vincula la cuenta del desarrollador con el registro del subdominio.

2. Mapeo de la IP Estática (Registro A)

En el panel de configuración de Duck DNS, se apuntó el subdominio directamente hacia la IP elástica pública asignada a nuestra instancia de AWS EC2. Esto crea un Registro de tipo A en los servidores DNS globales. A partir de ese momento, cualquier petición dirigida a `intellcar.duckdns.org` es enrutada de forma automática por la infraestructura de red hacia el puerto 80/443 de nuestra máquina virtual en AWS.

3. Sincronización Automática (Script en Cron)

Dado que las direcciones IP en entornos de nube pública pueden ser dinámicas ante reinicios severos de la instancia, se configuró una tarea programada interna (Cron job) en el sistema operativo de la máquina EC2.

Cada 5 minutos, el servidor ejecuta un comando de fondo mediante curl que notifica silenciosamente a la API de Duck DNS la IP actual del servidor:

```
echo "url=https://www.duckdns.org/update?domains=intellcar&token=${{ secrets.DUCKDNS_TOKEN }}" | curl -k -o /home/ubuntu/duckdns/duck.log -K -
```

Certificación SSL

Para garantizar la confidencialidad e integridad de los datos transmitidos entre los clientes (Angular / futuro React Native) y el servidor (Laravel API), es imprescindible sustituir el protocolo inseguro HTTP por HTTPS. Esto protege la plataforma frente a ataques de interceptación de datos (*Man-in-the-Middle*).

Para lograrlo a coste cero y bajo estándares de la industria, se ha implementado una infraestructura de clave pública utilizando la entidad certificadora Let's Encrypt y la herramienta de automatización Certbot.

1. El Proceso de Validación "HTTP-01 Challenge"

Como se observa en el flujo de GitHub Actions, el pipeline delega en Certbot la tarea de negociar el certificado con los servidores de Let's Encrypt de forma no interactiva:

```
sudo certbot --nginx -d ${{ secrets.DOMAIN_NAME }} --non-interactive  
--agree-tos --email ${{ secrets.USER_EMAIL }}
```

Cuando este comando se ejecuta dentro de la instancia EC2, ocurren las siguientes acciones en segundo plano:

Petición del Certificado: Certbot se comunica con Let's Encrypt solicitando un certificado para `intellcar.duckdns.org`.

El Desafío (Challenge): Let's Encrypt genera un token secreto y le pide a Certbot que lo aloje en una ruta específica de nuestro servidor web Nginx.

Verificación Global: Los servidores de Let's Encrypt realizan una petición HTTP hacia nuestro dominio. Al comprobar que Nginx responde con el token correcto, validan instantáneamente que nosotros poseemos y controlamos la máquina vinculada a ese dominio.

2. Mutación Automatizada de Nginx

Una vez superado el desafío, Certbot descarga los archivos de la clave pública (`fullchain.pem`) y la clave privada (`privkey.pem`) en el directorio `/etc/letsencrypt/live/`.

Lo más eficiente de esta solución es que Certbot modifica dinámicamente el archivo de configuración de Nginx que creamos en el pipeline:

Abre el puerto de escucha segura 443 (`listen 443 ssl`).

Inyecta las rutas hacia las claves criptográficas generadas.

Configura los parámetros de cifrado fuerte (TLS 1.2 / TLS 1.3).

Escribe una regla de redirección 301 para que cualquier usuario que intente entrar por el puerto 80 (HTTP) sea desviado de forma automática y obligatoria hacia el entorno seguro (HTTPS).

3. Automatización de la Renovación (Cron/Systemd)

Los certificados de Let's Encrypt son extremadamente seguros porque caducan a los 90 días, lo que reduce el margen de maniobra de un atacante que intente descifrar las claves.

Para evitar tener que renovarlo manualmente, la instalación de Certbot en la máquina Ubuntu de AWS EC2 crea un temporizador de sistema (Systemd timer o Cron job) que se ejecuta dos veces al día de forma silenciosa. Este servicio comprueba si al certificado le quedan menos de 30 días para expirar; si es así, realiza la renovación en caliente y recarga Nginx automáticamente sin interrumpir el servicio de Intellcar.

Con la implementación del subdominio en Duck DNS y el certificado SSL/TLS de Let's Encrypt, la arquitectura de Intellcar queda blindada en producción. El ecosistema completo es ahora capaz de comunicarse de forma segura con los servicios de Inteligencia Artificial de AWS (Rekognition y Comprehend) cumpliendo estrictamente con las normativas vigentes de privacidad y protección de datos.

6 — Conclusiones.

6.3 Conclusiones Técnicas

Tras la culminación del diseño, desarrollo e implantación del ecosistema Intellcar, se extraen conclusiones técnicas de alto valor que justifican la viabilidad y robustez de la plataforma. El proyecto no ha sido concebido meramente como una aplicación funcional de fin de ciclo, sino como una solución de software orientada a entornos de producción reales.

A continuación, se exponen los pilares de ingeniería de software por los cuales Intellcar destaca cualitativamente frente a otras propuestas académicas o comerciales estándar:

1. Auditoría Automatizada y Gobernanza Activa (AWS Rekognition y Comprehend)

La gran mayoría de las plataformas actuales delegan la seguridad del contenido en revisiones manuales reactivas (paneles de administración tradicionales) o, en el peor de los casos, carecen por completo de filtros, asumiendo riesgos legales y reputacionales críticos.

Intellcar destaca por introducir un paradigma de Seguridad por Diseño (Privacy and Security by Design):

Eficiencia frente al factor humano: Al integrar de forma asíncrona AWS Rekognition (detectModerationLabels, detectText) y de forma síncrona AWS Comprehend (detectPiiEntities, detectToxicContent), la plataforma no "espera" a que un administrador valide un post. La moderación ocurre en milisegundos las 24 horas del día.

Optimización del modelo de negocio: El uso de IA garantiza que los recursos de almacenamiento de la plataforma (Buckets S3) contengan única y exclusivamente imágenes lícitas y directamente vinculadas a la automoción (REQUIRED_VEHICLE_LABELS), protegiendo la base de datos de ataques de inyección de contenido basura (spam) y blindando legalmente al propietario del servicio.

2. Madurez en Operaciones y Automatización Absoluta (Pipeline CI/CD)

El archivo de orquestación `deploy.yml` implementado mediante GitHub Actions representa uno de los puntos con mayor carga de ingeniería del proyecto. Mientras que los despliegues académicos suelen basarse en subidas manuales por FTP o clonados directos via Git expuestos en terminal, Intellcar implementa un flujo de Integración y Despliegue Continuos (CI/CD) completamente profesional y desacoplado:

Estrategia Híbrida y Eficiente: El pipeline separa inteligentemente el empaquetado del cliente web (compilado eficientemente mediante `pnpm` y `Turborepo` para ser distribuido en AWS S3) de la lógica del backend.

Orquestación Atómica en la Nube: A través de una conexión SSH automatizada, el pipeline toma el control de la instancia EC2, genera configuraciones de entorno `.env` seguras sobre

la marcha y levanta la infraestructura sin provocar caídas de servicio. La automatización de optimizaciones críticas del framework (como el cacheado automático de rutas y configuraciones de Laravel junto a la autogeneración de la documentación Swagger) demuestra un entendimiento profundo del ciclo de vida del software en producción.

3. Infraestructura Cloud Sólida, Desacoplada y Elástica (AWS Ecosystem)

El diseño de la infraestructura de red en Amazon Web Services aleja a Intellcar del clásico patrón monolítico inestable (donde el servidor web, el código y la base de datos compiten por los recursos de una misma máquina).

Persistencia Delegada: La API de Laravel se ejecuta dentro de un contenedor Docker inmutable en AWS EC2, pero la persistencia de datos vive en un nodo gestionado e independiente de AWS RDS (MySQL) bajo una red privada virtual (VPC). Esto garantiza que un pico de tráfico en el servidor web jamás corrompa o degrade la velocidad de la base de datos.

Seguridad y Coste Cero: La capa de transporte se ha resuelto con brillantez y eficiencia. Utilizando herramientas abiertas como Duck DNS para la resolución dinámica y Certbot (Let's Encrypt) para la mutación automatizada de las directivas HTTPS de Nginx, se ha conseguido una arquitectura cifrada de nivel bancario sin añadir costes operativos al presupuesto del proyecto.

4. Visión de Futuro y Escalabilidad del Código (Arquitectura Monorepo)

La decisión de estructurar el Frontend como un monorepo gestionado por pnpm y Turborepo sitúa al proyecto en una posición de ventaja competitiva insuperable a nivel de desarrollo de producto:

El desacoplamiento de la lógica de negocio, tipados e interfaces de API dentro de la carpeta packages/ dota a la plataforma de una arquitectura modular.

El repositorio no es un sistema cerrado: está técnica y estructuralmente preparado para integrar de forma inmediata una aplicación móvil nativa en React Native en el directorio apps/mobile-app, la cual heredará todo el trabajo de integración cloud realizado para Angular, reduciendo los tiempos de comercialización (Time-to-Market) a más de la mitad.

En conclusión, *Intellcar* destaca porque sustituye los procesos artesanales y manuales por procesos automatizados de ingeniería. Es un ecosistema veloz en su interfaz de usuario (gracias a GSAP y Motion), inteligente y seguro en su backend (gracias a las soluciones serverless de IA de AWS) y robusto en su mantenimiento (gracias a Docker y las GitHub Actions). Cumple, de forma rigurosa, con todas las métricas de calidad que definen a una aplicación moderna de nivel corporativo.

6.3 Conclusiones Personales (José)

El desarrollo de IntellCar para mí no ha sido solo un 'trabajo' final de DAW, ha sido toda una demostración de la capacidad técnica que he logrado adquirir en todos estos años en los que me he estado formando como desarrollador de sistemas de software.

Sí, sistemas de software, creo que queda demostrado que 'hacer una web' es una definición que se queda muy floja y coja para el nivel al que he llegado gracias a este proyecto.

A diferencia de lo que he hecho en mis prácticas en centros de trabajo dónde he sido el 'Front-Developer' mayormente, (por mi trabajo con interfaces visuales con React Native y Expo, en los proyectos de un laboratorio de jóvenes emprendedores), aquí he sido todo lo contrario, he asumido el papel de gran parte del Backend y, sobre todo, de la infraestructura en la nube y los pipelines de despliegue. Esto ha hecho posible que me dé cuenta de la potencia que ofrece el implementar servicios de AWS como Rekognition, de qué se debe tener en cuenta en la seguridad, escalabilidad y mantenimiento de un proyecto, y en cómo debe ser automatizado todo de forma minuciosa para evitar errores humanos.

Para mí, un muchacho que logró entrar en DAW por los pelos más tarde que sus compañeros, tras no tener una experiencia satisfactoria en DAM y que casi se da por rendido, haber logrado concluir un proyecto que:

- Integra moderación con IA.
- Destaca por su infraestructura en la nube.
- Es capaz de desplegarse en pocos minutos con solo dos comandos.
- Funciona de forma muy fluida sin errores de ejecución.
- Tiene una estructura que perfectamente podría ser la de un sistema de cualquier empresa real...

Sinceramente, me parece alucinante, y más pensando que todo este salto lo he dado en menos de 6 meses (He pasado de no saber que PHP era un lenguaje mayoritario para servidores, a liderar y diseñar el sistema de este proyecto). Por ello, esto no lo considero 'un proyecto', como otros tantos del montón que he hecho, si no el trofeo merecido del increíble crecimiento exponencial que he experimentado en tan poco tiempo, y el cuál ha hecho tener clara mi orientación hacia la Ingeniería de Software con mentalidad DevOps y Cloud. El proyecto no es perfecto, soy plenamente consciente de ello, hay muchos aspectos que me hubiera gustado abordar de otra forma, incluso podrán comprobar que en el backend hay muchas funcionalidades que no se han llegado a implementar por cambios de planes para centrarnos en cosas más importantes.

Pero creo que se trata de un buen golpe en la mesa para demostrar que esto no es 'un juego', es algo serio, y puede que solo el comienzo de algo más grande...

6.3 Conclusiones Personales (Juan)

Desarrollar IntellCar ha sido, sin duda, la experiencia técnica más completa de mi formación como desarrollador web. Afrontar desde cero un proyecto full-stack real —con sus decisiones de arquitectura, sus imprevistos y sus iteraciones— me ha dado una perspectiva que ningún ejercicio aislado podría ofrecer.

En el plano técnico, el mayor aprendizaje ha venido de diseñar una **API REST escalable con Laravel 12** que soporte dominios tan distintos como un marketplace, una red social y un sistema de eventos, manteniéndola cohesionada bajo un único modelo de autenticación (Sanctum) y documentada con OpenAPI/Swagger. Aprendí que la arquitectura no es un lujo inicial: cada decisión de diseño se paga o se cobra con creces durante el desarrollo.

El trabajo con **Angular 19 en arquitectura standalone** dentro de un monorepo Turborepo me obligó a pensar en componentes reutilizables, gestión de estado y rendimiento desde el primer día. La integración con el backend mediante interceptores HTTP y guards de ruta fue uno de los puntos más desafiantes y, a la vez, más satisfactorios del proyecto.

Más allá de la técnica, este proyecto me ha enseñado a **gestionar la incertidumbre**: a priorizar funcionalidades, a tomar decisiones con información incompleta y a entregar valor incremental en lugar de perseguir la perfección. Son habilidades que, estoy convencido, marcarán mi carrera profesional desde el primer día de trabajo.

7. Futura Expansión del Proyecto

IntellCar está diseñado desde el principio para escalar. La API REST stateless, la separación entre frontend y backend, y la infraestructura en contenedores (Docker / AWS) permiten incorporar nuevas capas sin reescribir lo existente. A continuación se detallan las tres líneas de expansión planificadas para las próximas versiones.

v1.1

App móvil en React Native

Desarrollar una aplicación nativa para iOS y Android que consuma la misma API REST ya existente. React Native permite compartir lógica de negocio entre plataformas y reducir el tiempo de desarrollo. Las funcionalidades prioritarias para la app serán las de mayor uso en movilidad: **exploración del Marketplace** (con filtros por Paddock), **notificaciones push** de Kdds y actividad social, y **gestión del garaje personal** con cámara integrada para subir fotos de vehículos. La app se autenticará mediante los mismos tokens Bearer de Sanctum que usa el cliente web, sin necesidad de cambios en el backend.

v1.2

Pasarela de pago — Stripe

Integrar **Stripe** como pasarela de pago para activar los flujos de monetización definidos en el plan de empresa. Se implementarán tres productos:

- **Anuncios destacados:** pago único por anuncio (checkout de una sola vez).

- **Suscripción Premium:** cobro recurrente mensual/anual mediante Stripe Billing y webhooks para gestionar altas, bajas y renovaciones automáticamente.
- **Patrocinio de Kdds:** pago manual con Stripe Payment Links para acuerdos con marcas y organizadores de eventos.

v2.0

Versión PRO — funcionalidades avanzadas

La versión 2.0 incorporará capacidades que transforman IntellCar de plataforma de contenido en una herramienta inteligente:

- **Motor de recomendación:** algoritmo basado en el historial del garaje, los Paddocks del usuario y sus interacciones sociales para sugerir anuncios, Kdds y perfiles afines (ML con Python, expuesto como microservicio REST).
- **Chat en tiempo real:** mensajería privada entre usuarios interesados en un anuncio, implementada con **WebSockets** (Laravel Reverb o Pusher) y sincronizada con el sistema de notificaciones ya existente.
- **Panel Pro Dealer:** dashboard para concesionarios con métricas de anuncios (vistas, contactos, tasa de conversión) y herramientas de gestión de inventario en lote (importación CSV / API).
- **SEO dinámico:** generación de landing pages estáticas por marca y modelo (Angular SSR / pre-rendering) para captar tráfico orgánico.

Versión	Funcionalidad	Tecnología	Estado
v1.0	API REST completa, auth, marketplace, garaje, social, eventos	Laravel 12 + Angular 19 + MySQL	✓ Completada
v1.1	App móvil nativa	React Native + misma API	Planificada
v1.2	Pagos y suscripciones	Stripe Billing + Webhooks	Planificada
v2.0	Recomendación ML, chat en tiempo real, panel dealer, SSR	Python ML · Laravel Reverb · Angular SSR	Hoja de ruta

Anexos

Anexo I — Plan de Empresa Completo (IPE)

Documento elaborado en el módulo de **Iniciativa Emprendedora (IPE)**. Recoge el análisis completo del modelo de negocio de IntellCar: estudio de mercado detallado, análisis de la competencia, plan de marketing y comunicación, plan operativo, forma jurídica (constitución de S.L., trámites en OEPM, obligaciones fiscales) y plan financiero proyectado a 3 años. El resumen ejecutivo de dicho documento se incluye en el apartado **2.2** de esta memoria.

[Plan de Empresa - IntellCar](#)